



哈爾濱工業大學

HARBIN INSTITUTE OF TECHNOLOGY



RESEARCH CENTER

# 网络通信基础 (I)

## TCP可靠传输与拥塞控制机制

王宏志

哈尔滨工业大学

计算学部 海量数据计算研究中心

wangzh@hit.edu.cn

<http://homepage.hit.edu.cn/wang>

# 目录

## Contents

- 1 UDP
- 2 TCP
- 3 拥塞控制
- 4 TCP 的演变
- 5 TCP 的问题

# 目录

## Contents

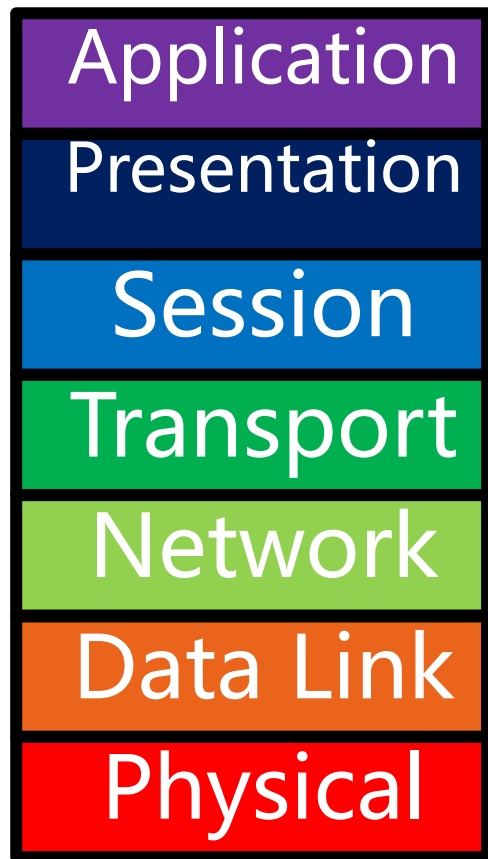
1 UDP

2 TCP

3 拥塞控制

4 TCP 的演变

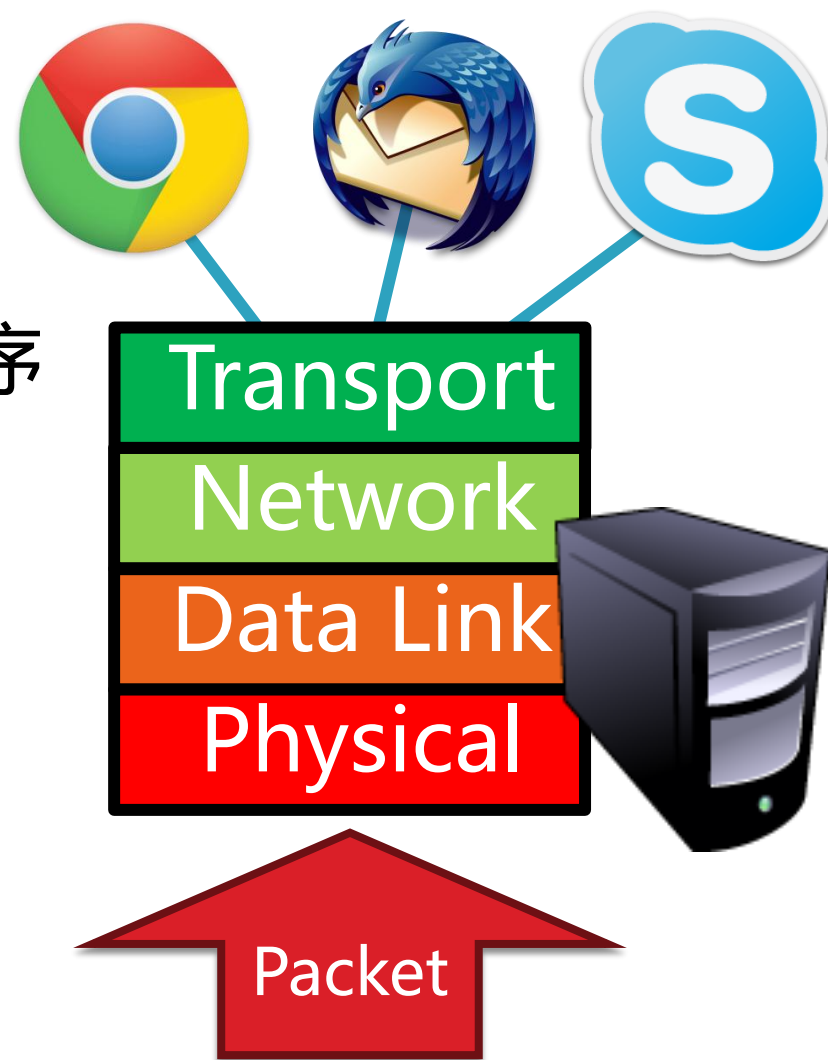
5 TCP 的问题



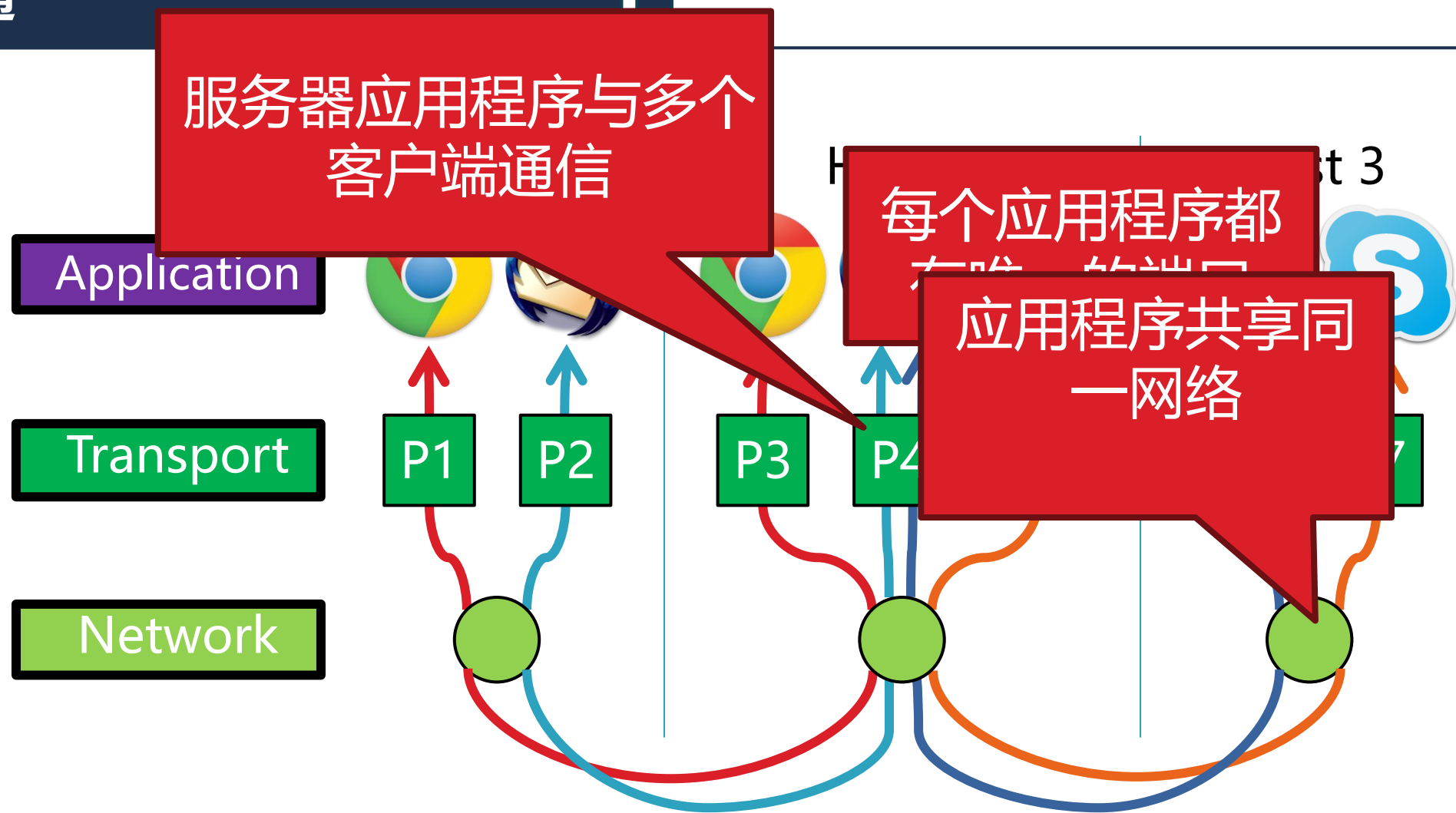
- 功能:
  - ▣ 数据流的解复用
- 可选功能:
  - ▣ 建立长久的联系
  - ▣ 可靠、按顺序递送数据包
  - ▣ 错误检测
  - ▣ 流量和拥塞控制
- 主要挑战:
  - ▣ 检测和应对拥塞
  - ▣ 平衡公平与高利用率

## 多路复用的案例

- 数据报网络
  - ▣ 没有电路
  - ▣ 没有连接
- 客户端同时运行许多应用程序
  - ▣ 要给谁投递packet?
- IP 标头"协议"字段
  - ▣ 8 位 = 256 个并发流
- 插入传输层以处理解复用

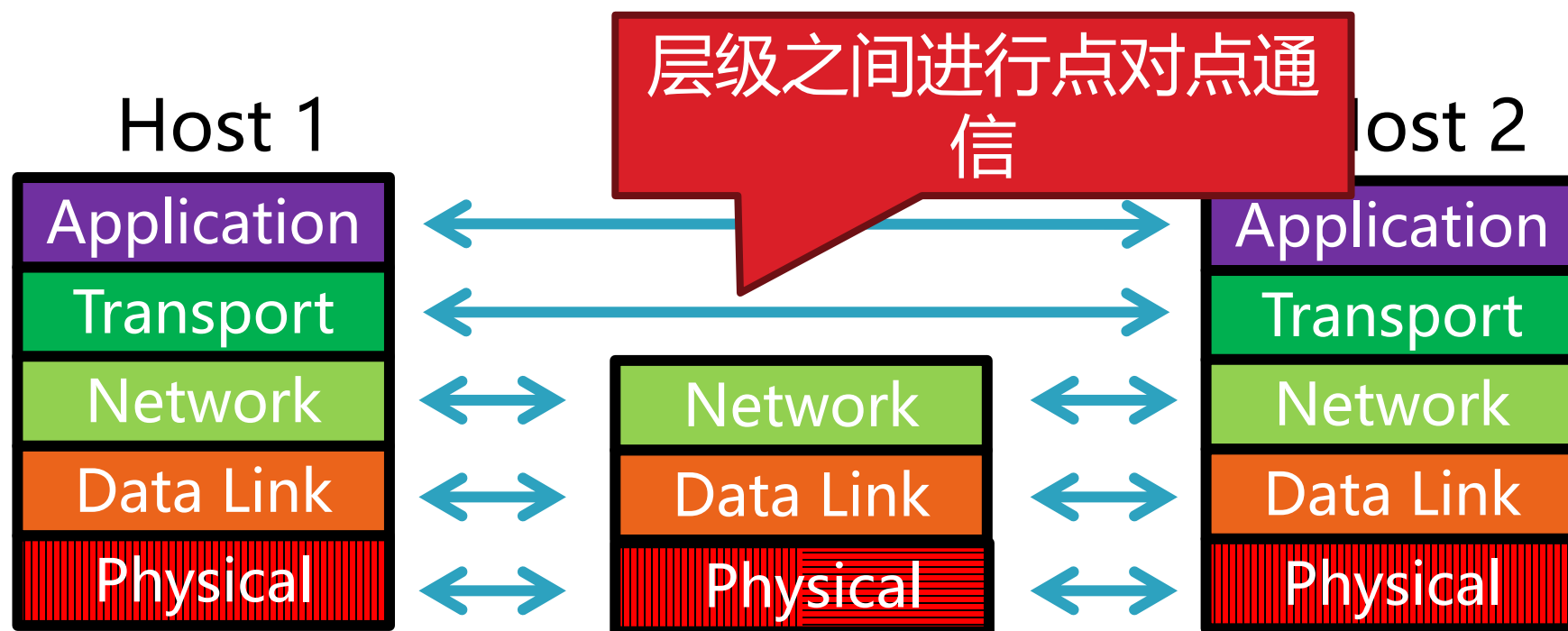


# 解复用交通



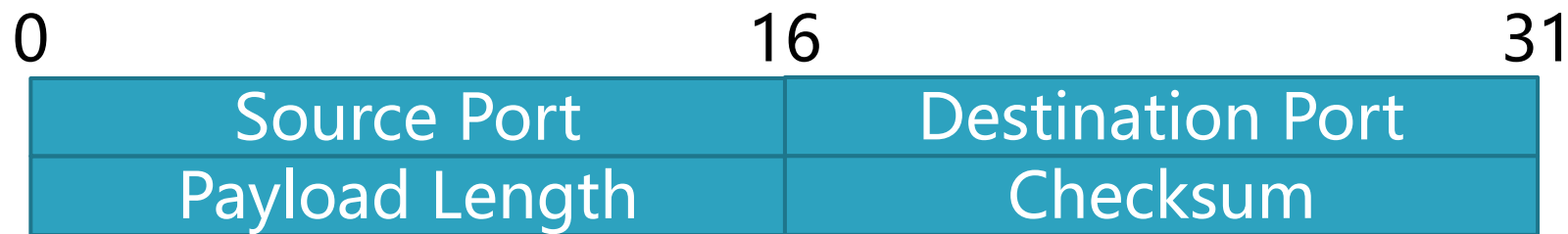
所识别的端点  $\langle src\_ip, src\_port, dest\_ip, dest\_port \rangle$

## 分层,重温



- 最低级别的端到端协议
  - ▣ 传输头仅由源和目标读取
  - ▣ 路由器将传输标头视为有效负载

# 用户数据报协议 (UDP)



- 简单，无连接的数据报
  - ▣ C套接字：SOCK\_DGRAM
- 端口号用于解复用
  - ▣ 16位 = 65535个可能的端口
  - ▣ 端口号0无效
- 校验和用于错误检测
  - ▣ 检测（一些）损坏的包
  - ▣ 不检测丢失、重复或顺序错乱的包

# UDP 的用途

- 在TCP之后发明
- 为什么?
- 并非所有应用程序都能容忍 TCP
- 自定义协议可以构建在 UDP 之上
  - 可靠性? 严格的顺序?
  - 流量控制? 拥塞控制?
- 例子
  - RTMP,实时媒体流(例如语音、视频)
  - Facebook 数据中心协议

# 目录

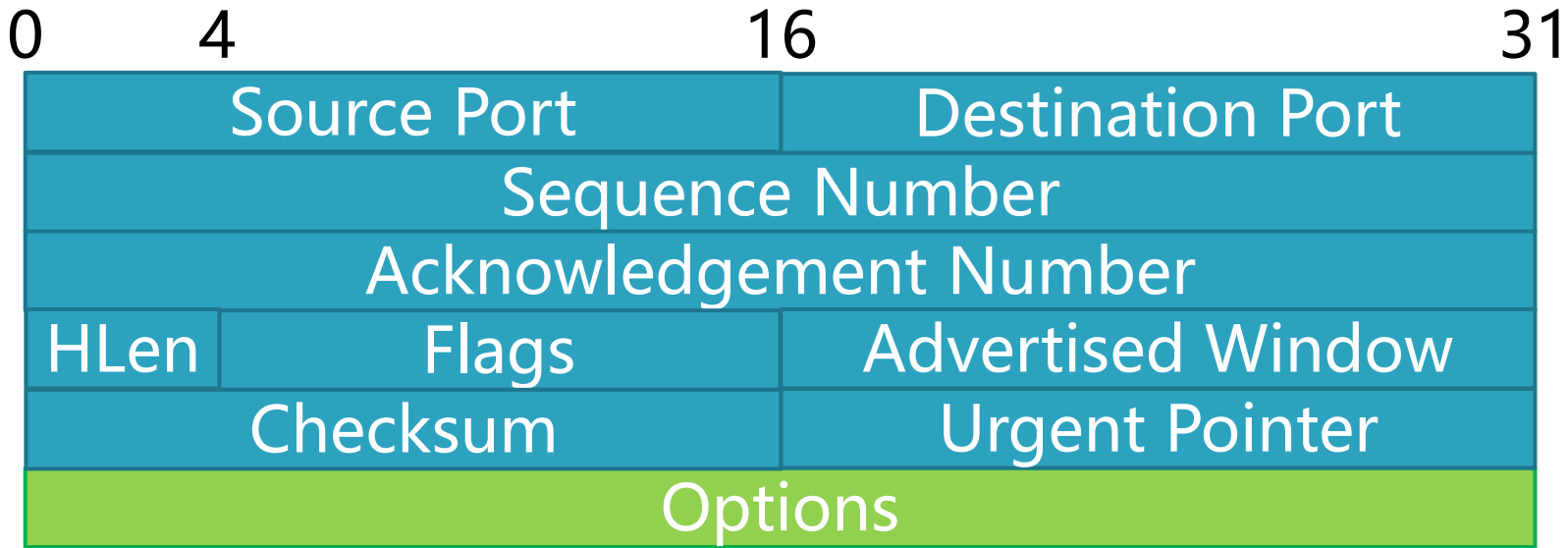
## Contents

- 1 UDP
- 2 TCP**
- 3 拥塞控制
- 4 TCP 的演变
- 5 TCP 的问题

# 传输控制协议

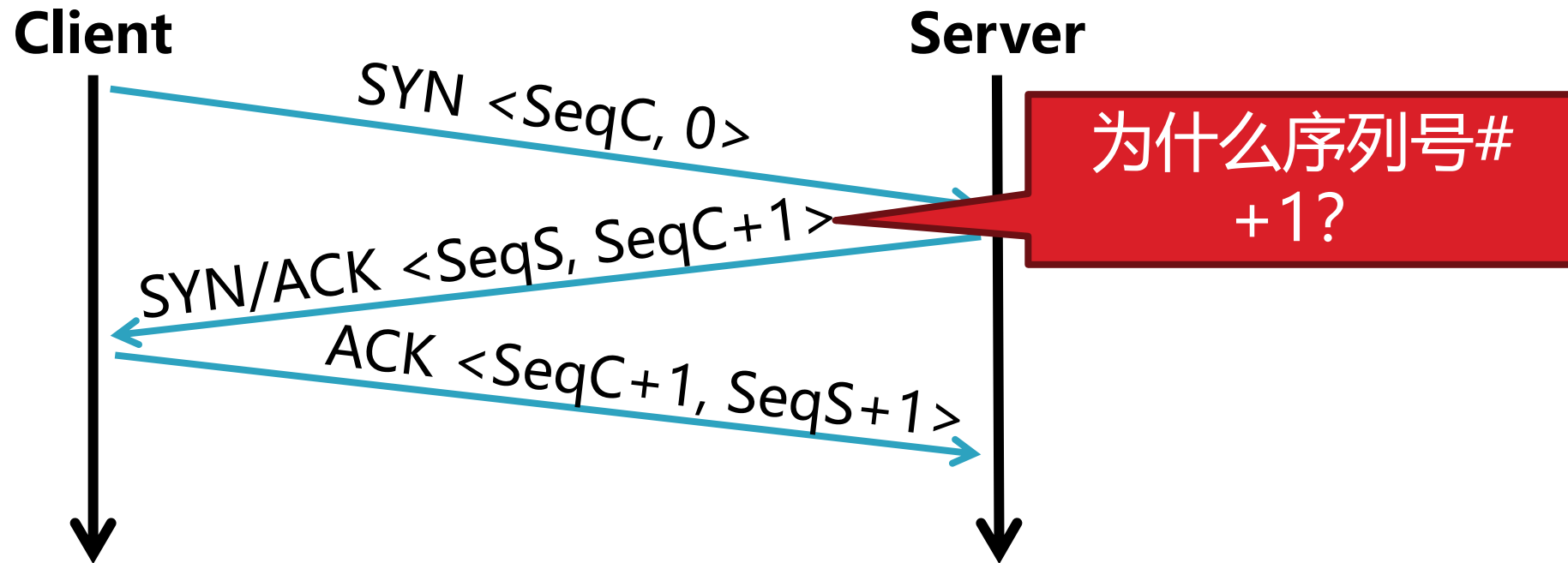
- ❑ 可靠、有序、双向字节流
- ❑ 解复用的端口号
- ❑ 虚拟电路(连接)
- ❑ 流量控制
- ❑ 拥塞控制,近似公平

为什么有这  
些功能?



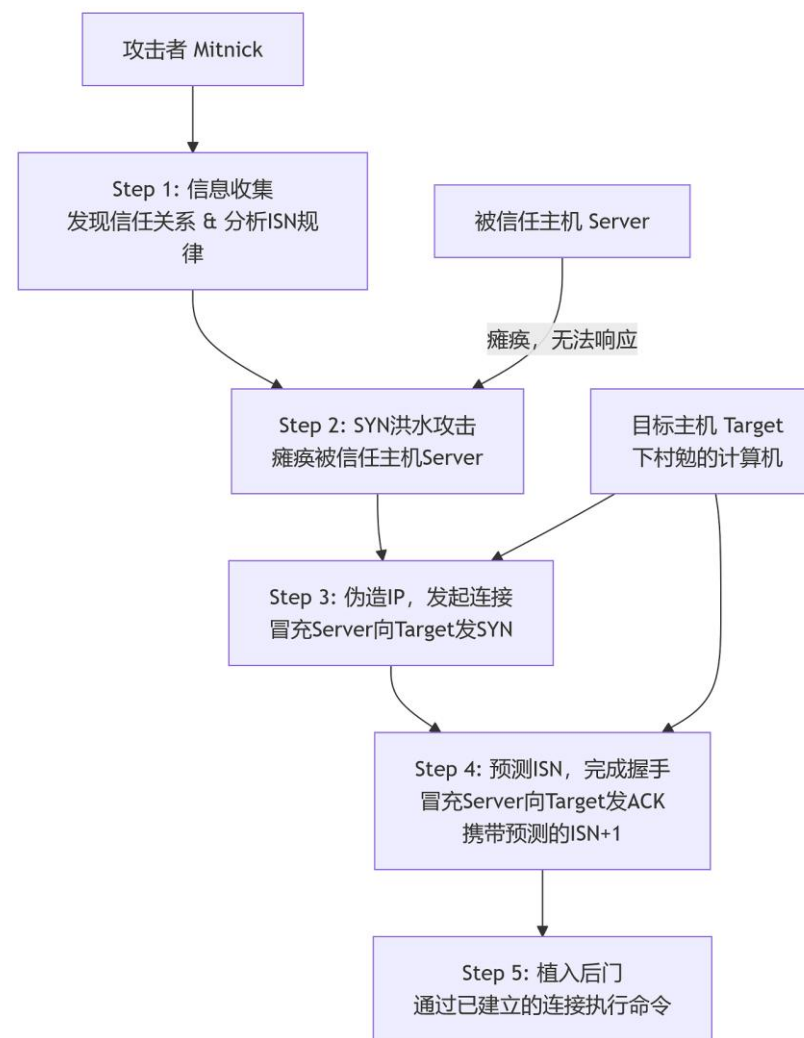
- 为什么需要建立连接?
  - ▣ 在两个主机上建立状态
  - ▣ 最重要的状态：序列号
    - 计算已发送的字节数
    - 随机选择的初始值
    - 为什么？
- 重要的 TCP 标志(每个 1 位)
  - ▣ SYN - 同步，用于连接建立
  - ▣ ACK - 确认接收到的数据
  - ▣ FIN - 完成，用于断开连接

# 三次握手



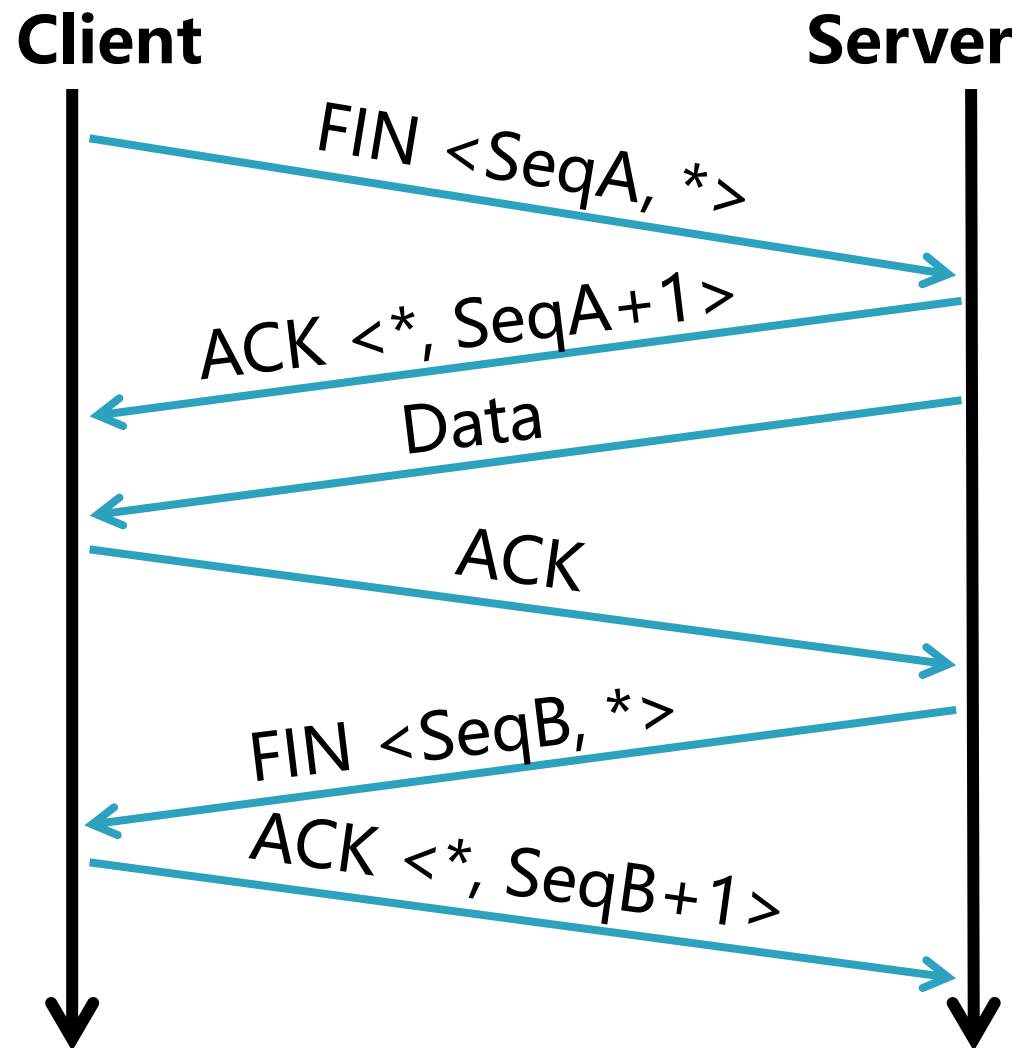
- 每次:
- 通知对方起始序号
- ACK对方的起始序号

- ❑ 连接混乱
  - ▣ 如何消除同一主机的联系?
  - ▣ 随机序号
- ❑ 来源欺骗
  - ▣ Kevin Mitnick
  - ▣ 需要好的随机数生成器!
- ❑ 连接状态管理
  - ▣ 每个 SYN 在服务器上分配状态
  - ▣ SYN flood = 拒绝服务攻击
  - ▣ 解决方案: SYN cookies

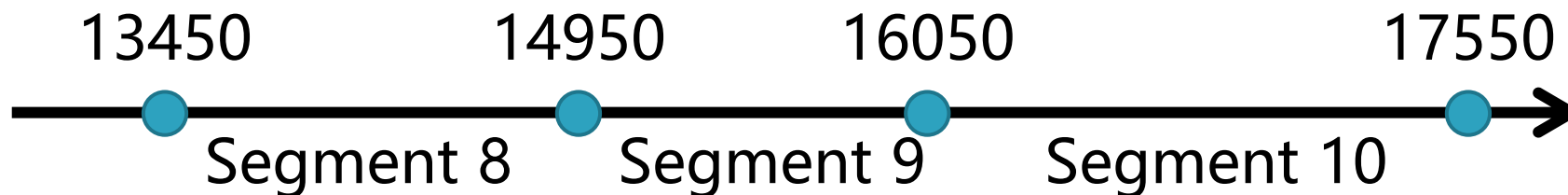


# 连接断开

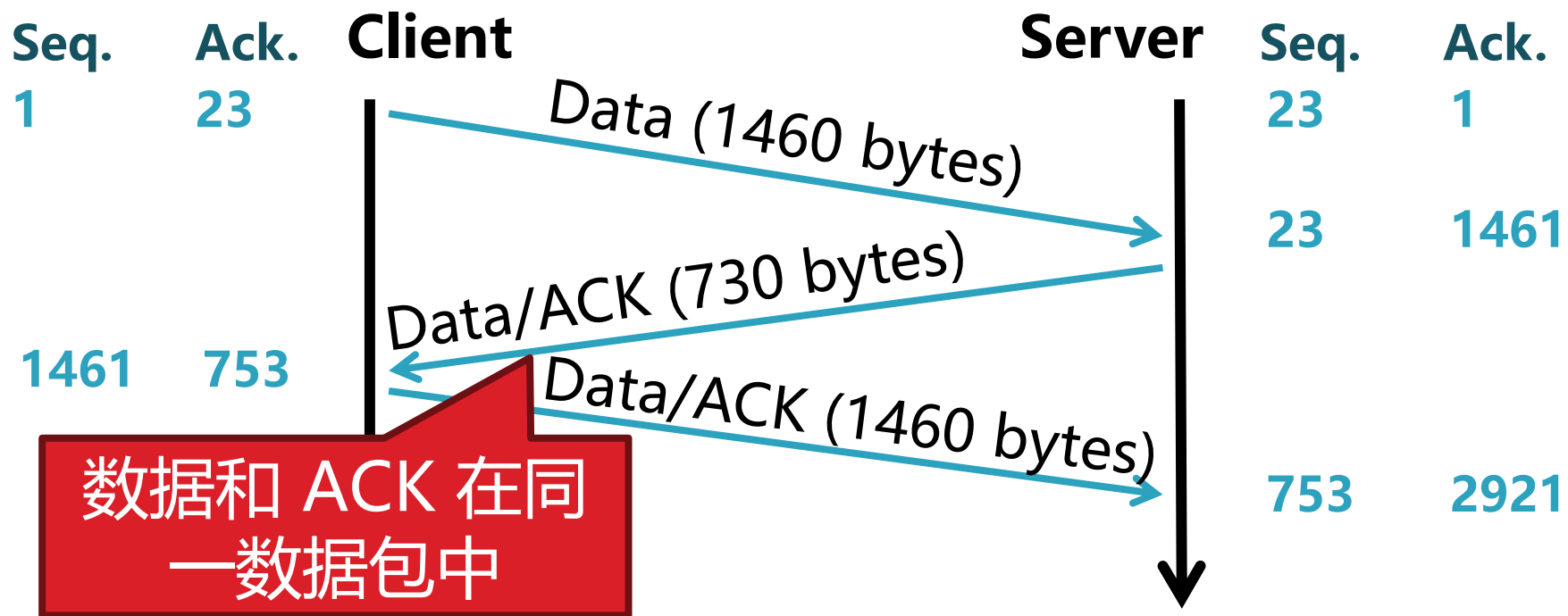
- 任何一方均可发起拆除
- 另一方可能继续发送数据
  - ▣ 半开连接
  - ▣ *shutdown()*
- 承认最后的 FIN
  - ▣ Sequence number + 1
- 如果第二个 FIN 丢失了怎么办?



- TCP 使用字节流抽象
  - ▣ 每个流中的每个字节都进行编号
  - ▣ 32位值，循环回绕
  - ▣ 初始，设置期间选定的随机值
- 字节流分解为segments (packets)
  - ▣ 大小受最大段大小限制(MSS)
  - ▣ 设置为限制碎片化
- 每个片段都有一个序号



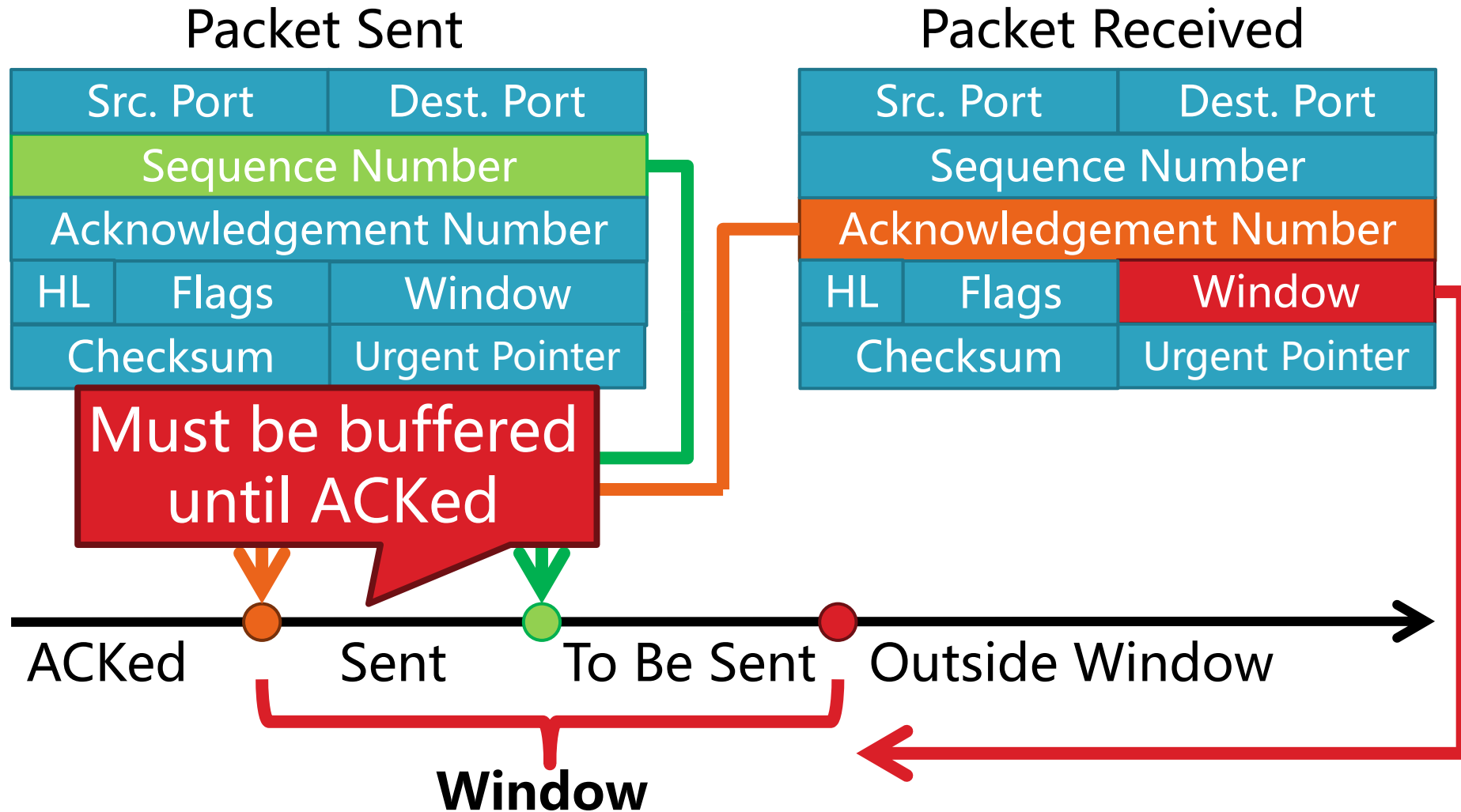
## 双向通信



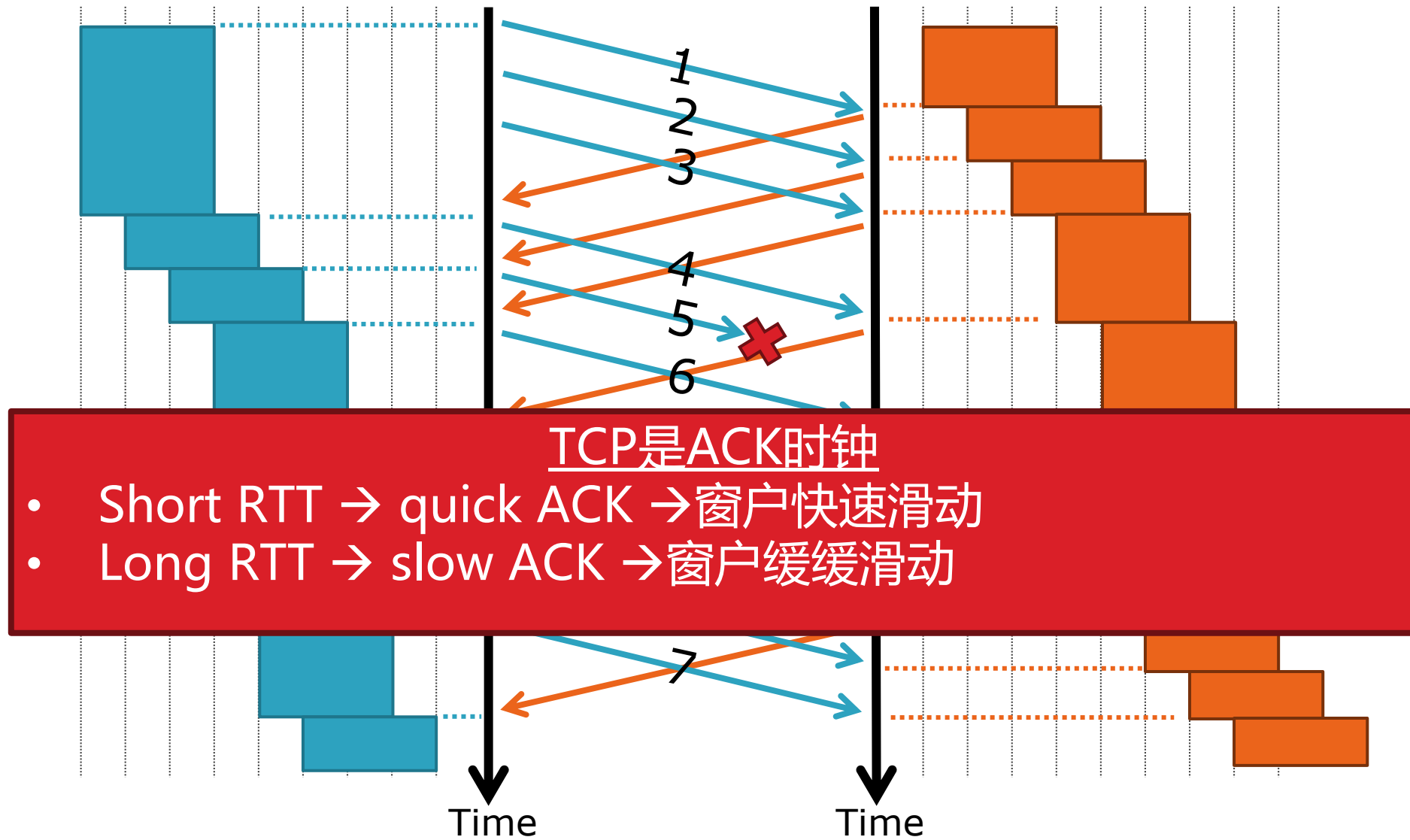
- 连接的每一侧都可以发送和接收
  - ▣ 每个方向的序号不同

- 问题:发送者应该发送多少个数据包?
  - ▣ 数据包太多可能会压倒接收者
  - ▣ 接收器缓冲区的大小可能会随着时间的推移而变化
- 解决方案: 滑动窗口
  - ▣ 接收者告诉发送者他们的缓冲区有多大
  - ▣ 称为广告窗口
  - ▣ 对于窗口大小  $n$ ,发送方可以发送  $n$  字节而不接收 ACK
  - ▣ 每 ACK 之后,窗户就会向前滑动
- 窗口可能会归零!

## 流量控制：发送端



## 滑动窗口示例



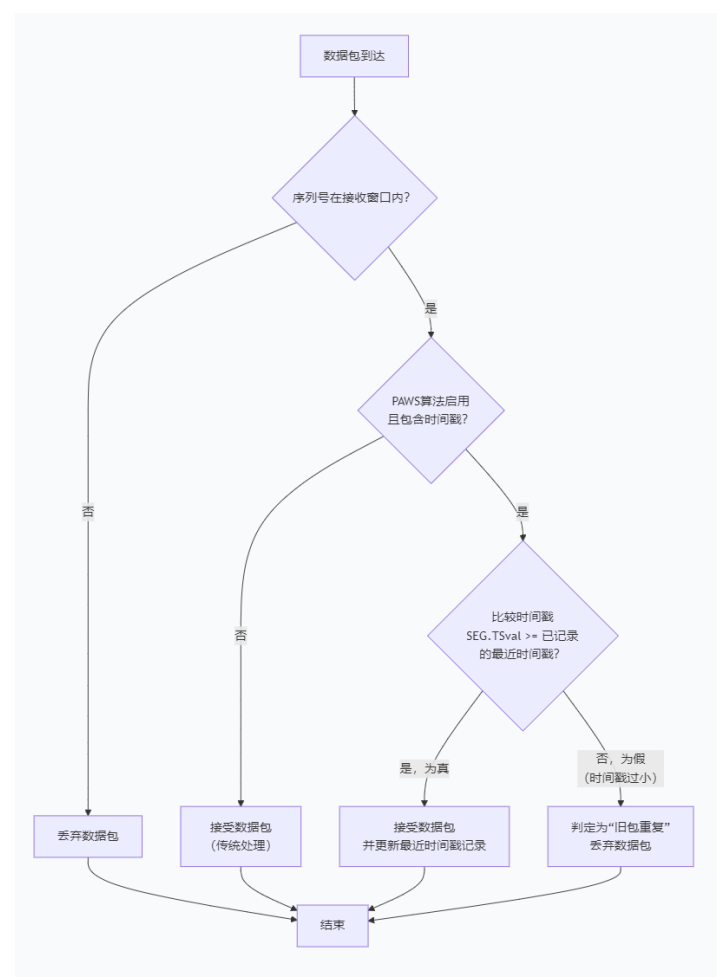
- 吞吐量为  $\sim w/\text{RTT}$
- 发送方必须缓冲所有未确认的数据包,因为它们可能需要重传
- 接收者可以接受无序数据包,但仅限于缓冲区限制

## 接受者回复什么?

1. ACK每包
2. 用累积 *ACK*, 式中, 序列  $n$  的ACK意味着所有  $k < n$  的ACKs
3. 用负 *ACK* (NACKs), 指示哪个包没有到达
4. 用选择性 *ACK* (SACKs), 指示那些已经到达的人, 即使不按顺序到达
  - ▣ SACK 是实际的 TCP 扩展

## 序号, 重访

- ❑ 32 bits, unsigned
  - ▣ 为什么这么大?
- ❑ 对于滑动窗口, 需要.....
  - ▣  $|\text{Sequence \# Space}| > 2 * |\text{Sending Window Size}|$
  - ▣  $2^{32} > 2 * 2^{16}$
- ❑ 防止乱码包
  - ▣ IP数据包的最大段生存时间 (MSL) 为120秒
    - 即一个数据包可以在网络中停留 2 分钟
  - ▣ 序号将以 286Mbps 绕行(怎么算出来的?)
    - 千兆以太网呢? PAWS 算法 + TCP 选项



- 问题:如果窗口尺寸很小怎么办?
- 多个小数据包、标头主宰数据



- 等价问题:发送者一次传输一个字节的数据包
  1. `for (int x = 0; x < strlen(data); ++x)`
  2. `write(socket, data + x, 1);`

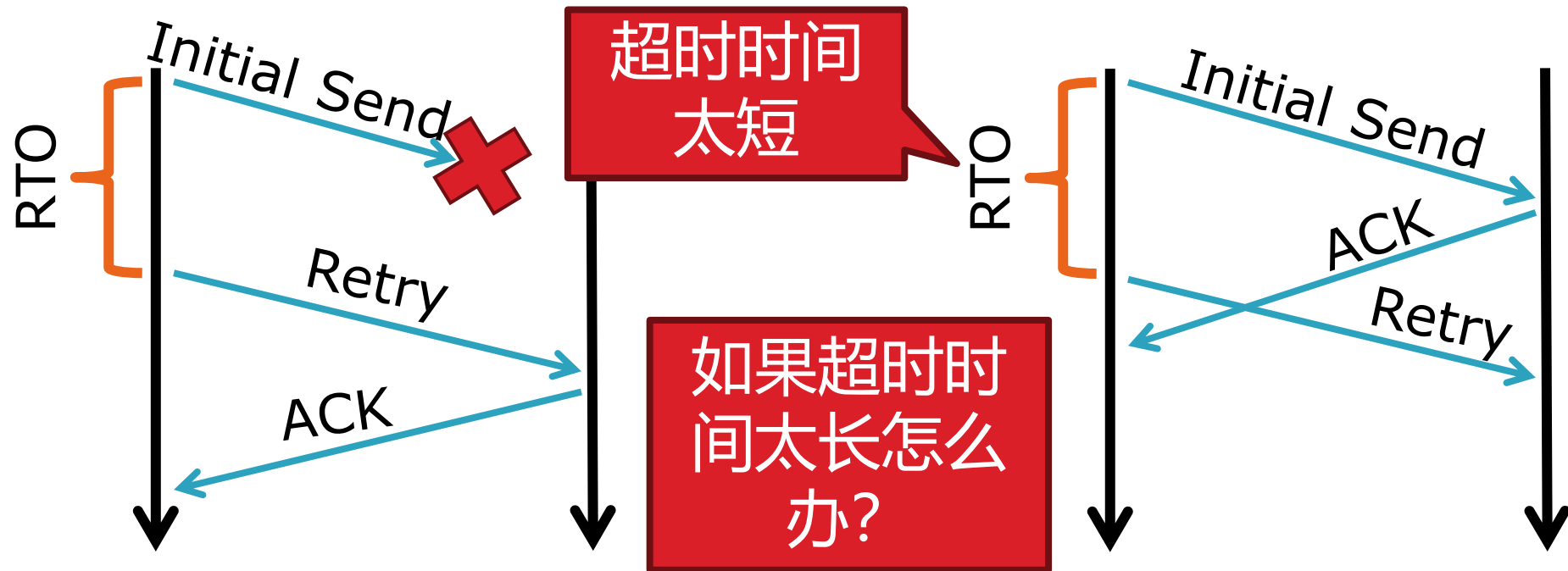
# Nagle 的算法

1. If the window  $\geq$  MSS and available data  $\geq$  MSS:  
Send the data 
  2. Elif there is unACKed data:  
Enqueue data in a buffer until an ACK is received
  3. Else: send the data 
- 问题:Nagle 的算法延迟传输
- 如果需要立即发送一个包裹怎么办?
    1. `int flag = 1;`
    2. `setsockopt(sock, IPPROTO_TCP, TCP_NODELAY, (char *) &flag, sizeof(int));`

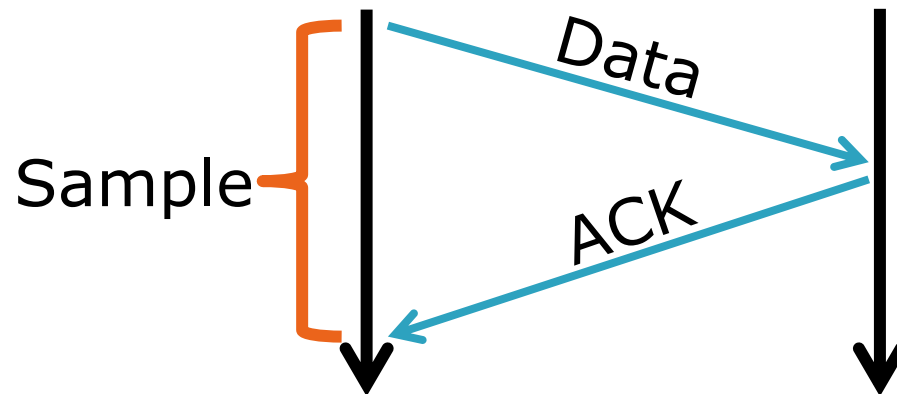
- ❑ 校验和检测(一些)数据包损坏
  - ▣ 通过 IP 标头、TCP 标头和数据计算
- ❑ 序号捕获序列问题
  - ▣ 重复者则忽略
  - ▣ 无序数据包被重新排序或丢弃
  - ▣ 缺少序列号表示丢失数据包
- ❑ 发送者检测到的丢失段
  - ▣ 使用超时来检测丢失的 ACK
  - ▣ 需要估计 RTT 以校准超时
  - ▣ 发送者必须保留所有数据的副本,直到 ACK

## 重传超时 (RTO)

- 问题:超时与往返时间有关

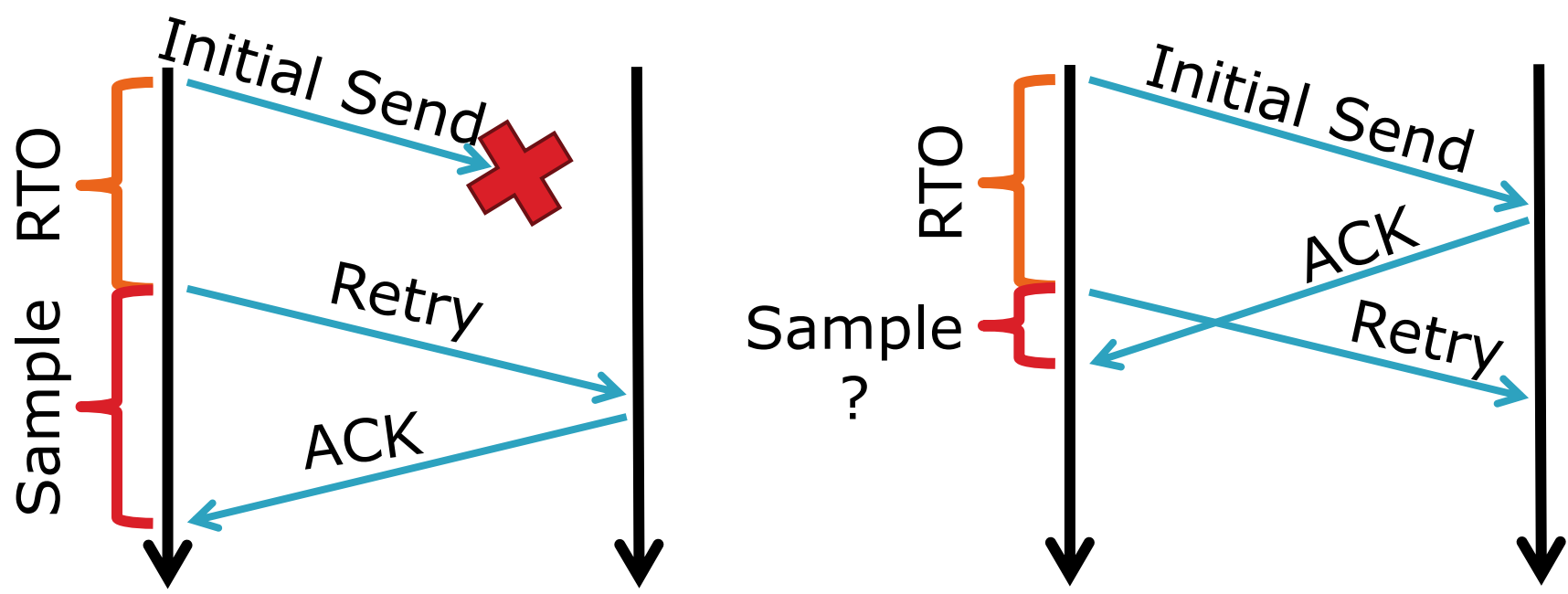


## 往返时间估计



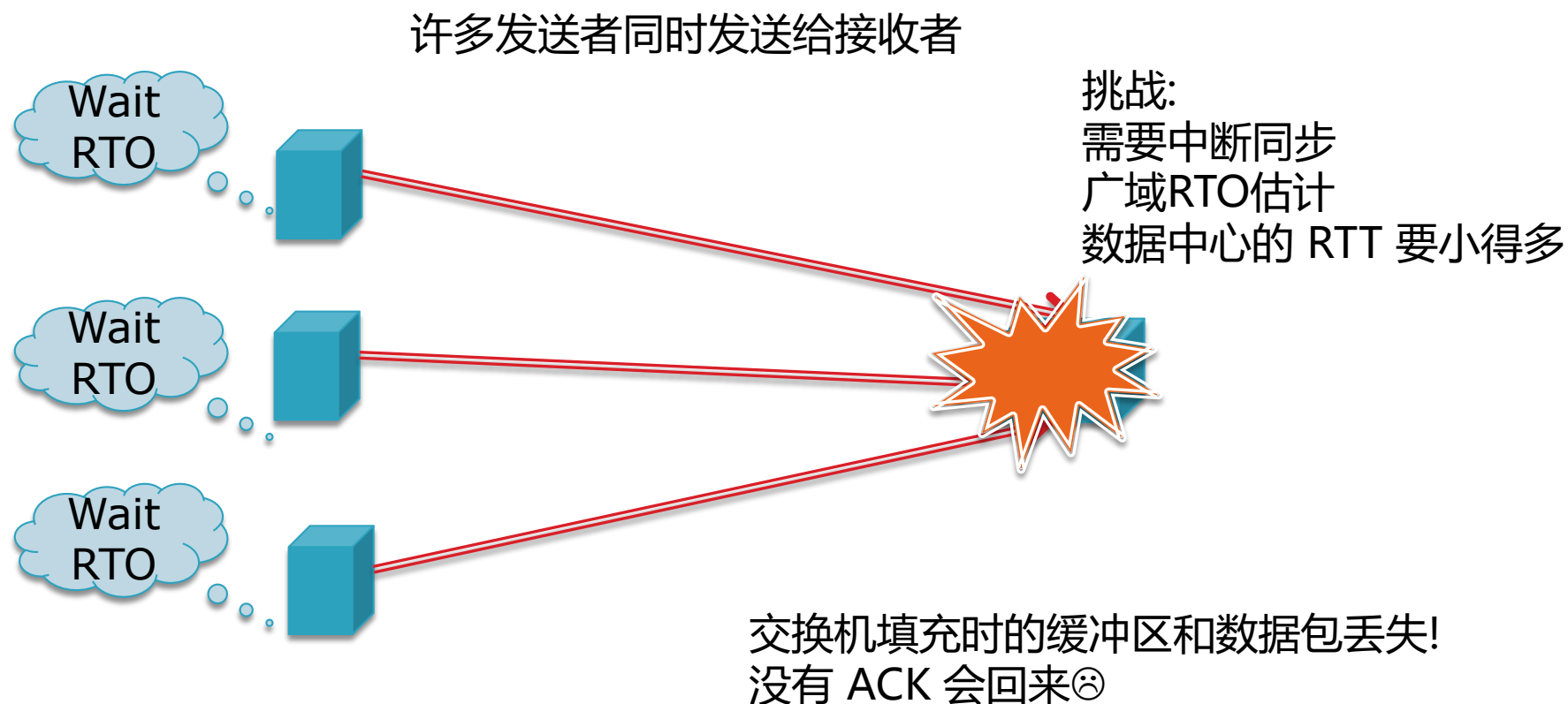
- 原始TCP往返时估算器
  - ▣ RTT 估计为移动平均线
  - ▣  $\text{new\_rtt} = \alpha (\text{old\_rtt}) + (1 - \alpha)(\text{new\_sample})$
  - ▣ 推荐 $\alpha$ :0.8-0.9(大多数TCP为0.875)
- $\text{RTO} = 2 * \text{new\_rtt}$  (TCP是保守的)

# RTT 样本歧义



□ Karn 算法:忽略重传片段的样本

## □ TCP Incast 问题 – 例如 Hadoop、Map Reduce、HDFS、GFS



# 目录

## Contents

1 UDP

2 TCP

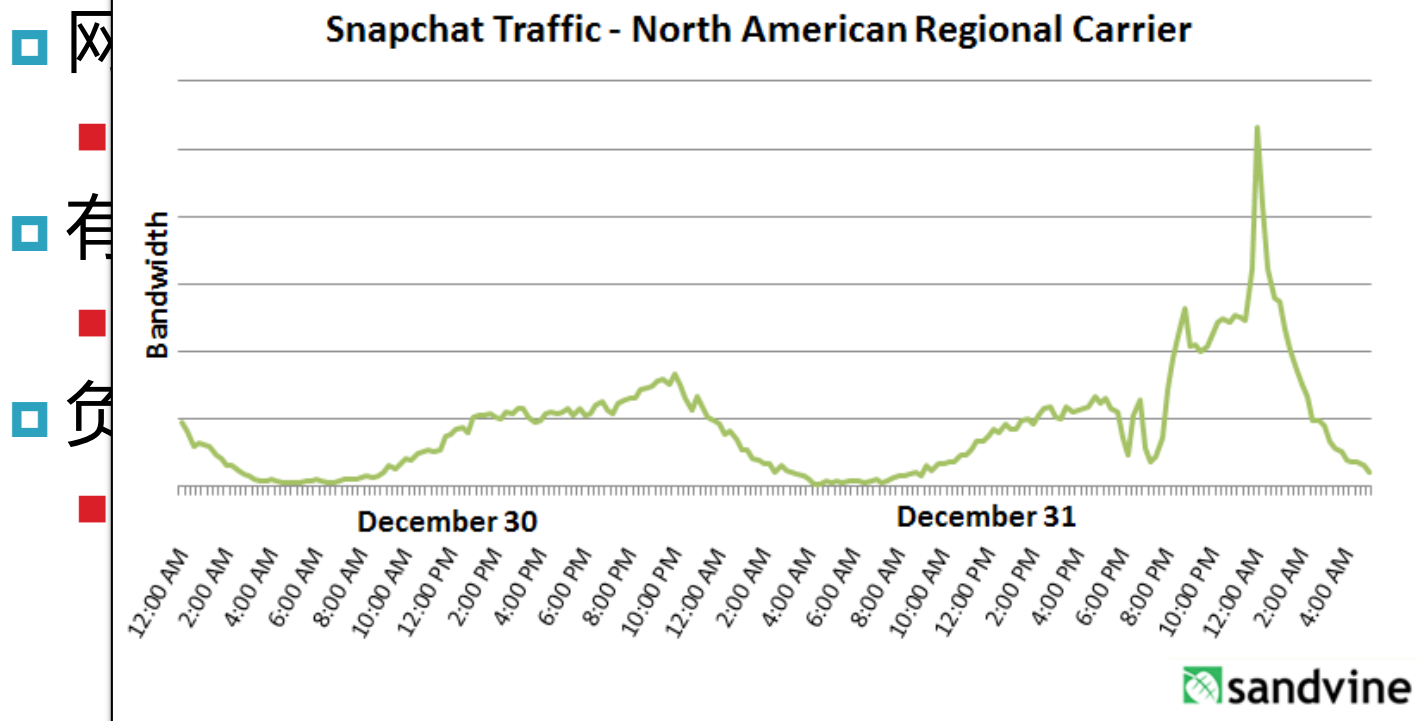
**3 拥塞控制**

4 TCP 的演变

5 TCP 的问题

# 什么是拥塞?

## □ 网络负载高于容量

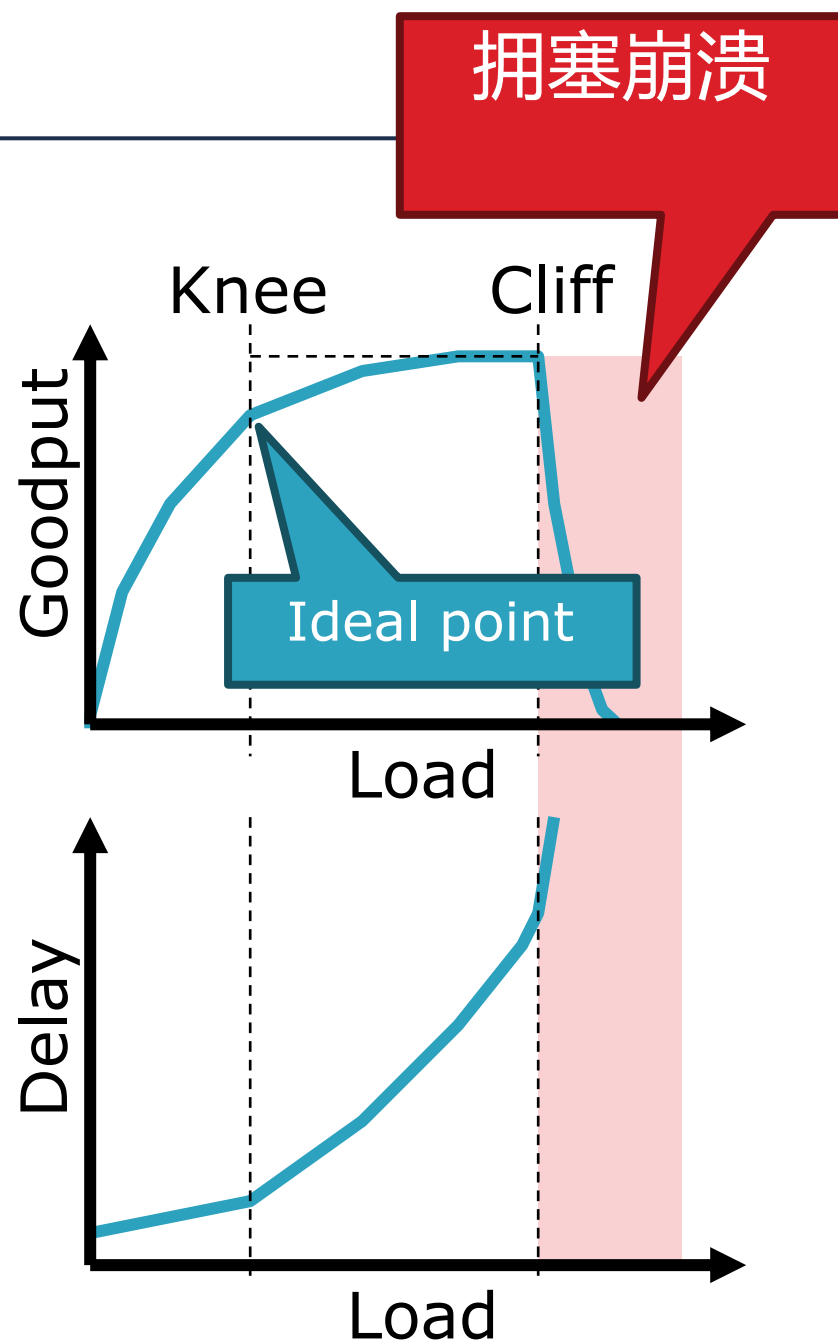


# 为什么拥塞不好?

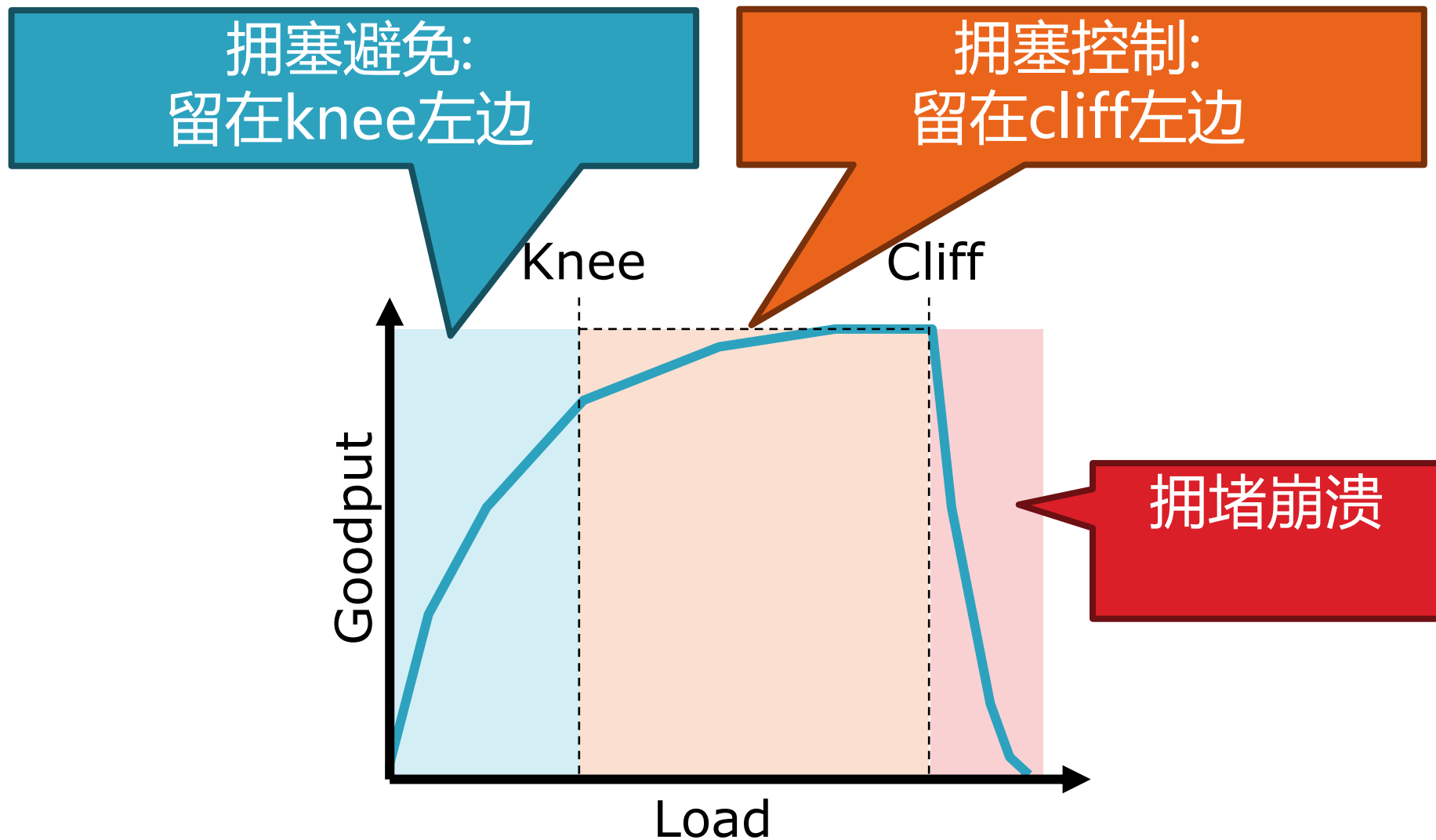
- ❑ 导致丢包
  - ▣ 路由器的缓冲区有限
  - ▣ Internet 流量是自相似的,没有缓冲区可以防止所有丢弃
  - ▣ 当路由器过载时,数据包将被丢弃
- ❑ 实际后果
  - ▣ 路由器队列堆积,延迟增加
  - ▣ 重传浪费带宽
  - ▣ 低网络"goodput"

## 增加负载的危险

- Knee – point之后
  - ▣ 吞吐量增长非常缓慢
  - ▣ 延迟增长很快
- 在 M/M/1 队列中
  - ▣  $\text{Delay} = 1/(1 - \text{utilization})$
- Cliff – point之后
  - ▣  $\text{Throughput} \rightarrow 0$
  - ▣  $\text{Delay} \rightarrow \infty$



## 拥塞控制vs.拥塞回避



- TCP 的广告窗口能解决拥塞问题吗?

NO

- 广告窗口仅保护接收者
- 足够快的接收器可以最大化窗口
  - ▣ 如果网络比接收器慢怎么办?
  - ▣ 如果有其他并发流怎么办?
- 要点
  - ▣ 窗口大小决定发送速率
  - ▣ 必须调整窗户以防止拥堵崩溃

## 拥堵控制的目标

1. 适应瓶颈带宽
2. 适应带宽的变化
3. 在流之间共享带宽
4. 最大化吞吐量

# 一般方法

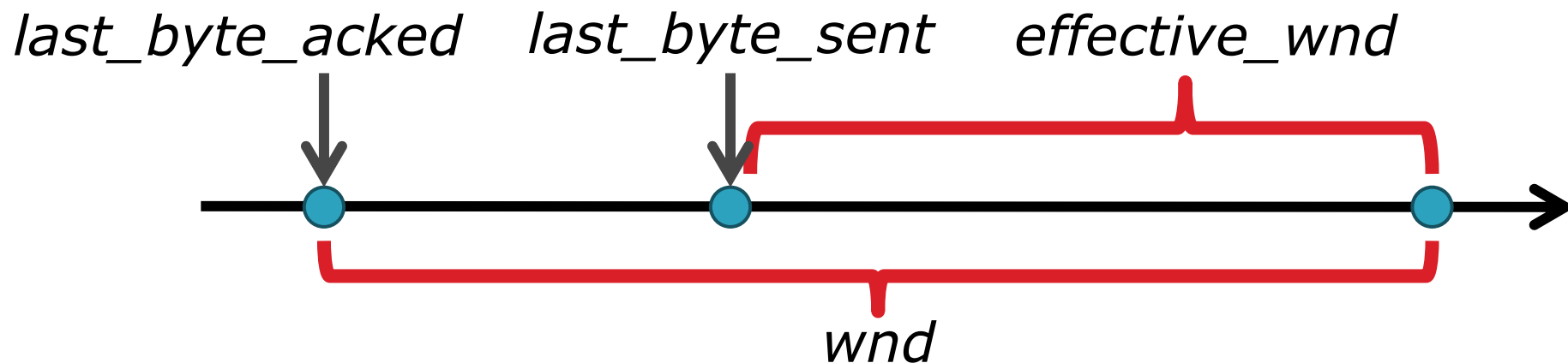
- ❑ 什么也不做，无差别发送数据包
  - ▣ 许多数据包将丢失，性能完全不可预测
  - ▣ 可能导致拥塞崩溃
- ❑ 预留
  - ▣ 为流量预先安排带宽分配
  - ▣ 发送数据包前需要协商
  - ▣ 必须得到网络的支撑
- ❑ 动态调整
  - ▣ 使用探测来估计拥塞程度
  - ▣ 拥塞低时加速
  - ▣ 拥塞增加时减速
  - ▣ 混乱的动态，需要分布式协调

- 每个TCP连接都有一个窗口
  - ▣ 控制未确认数据包的数量
  - ▣ 发送速率约为窗口/往返时间
- 想法：通过改变窗口大小来控制发送速率
- 在发送端引入拥塞窗口
  - ▣ 拥塞控制是发送端的问题

## 拥塞窗口 (cwnd)

- 限制传输中的数据量
- 以字节为单位

1.  $wnd = \min(cwnd, adv\_wnd);$
2.  $effective\_wnd = wnd - (last\_byte\_sent - last\_byte\_acked);$



## 两个基本组成部分

### 1. 检测拥堵

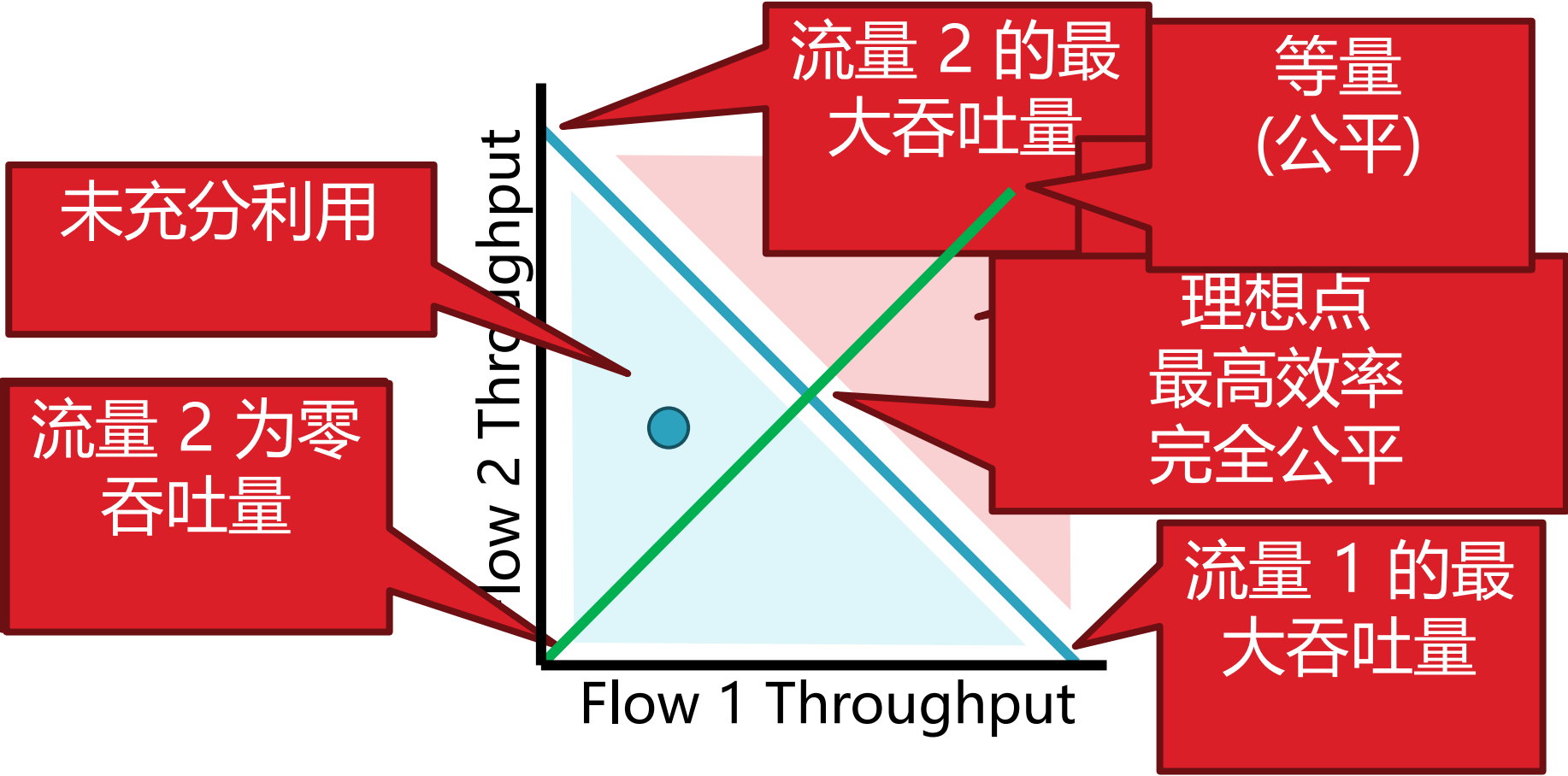
- 丢包是最可靠的信号
  - 基于延迟的方法既难又危险
- 如何检测丢包? ACK
  - 未收到 ACK 后暂停
  - 连续几个重复的 ACK(暂时忽略)

### 2. 速率调整算法

- 修改 cwnd
- 探测带宽
- 应对拥堵

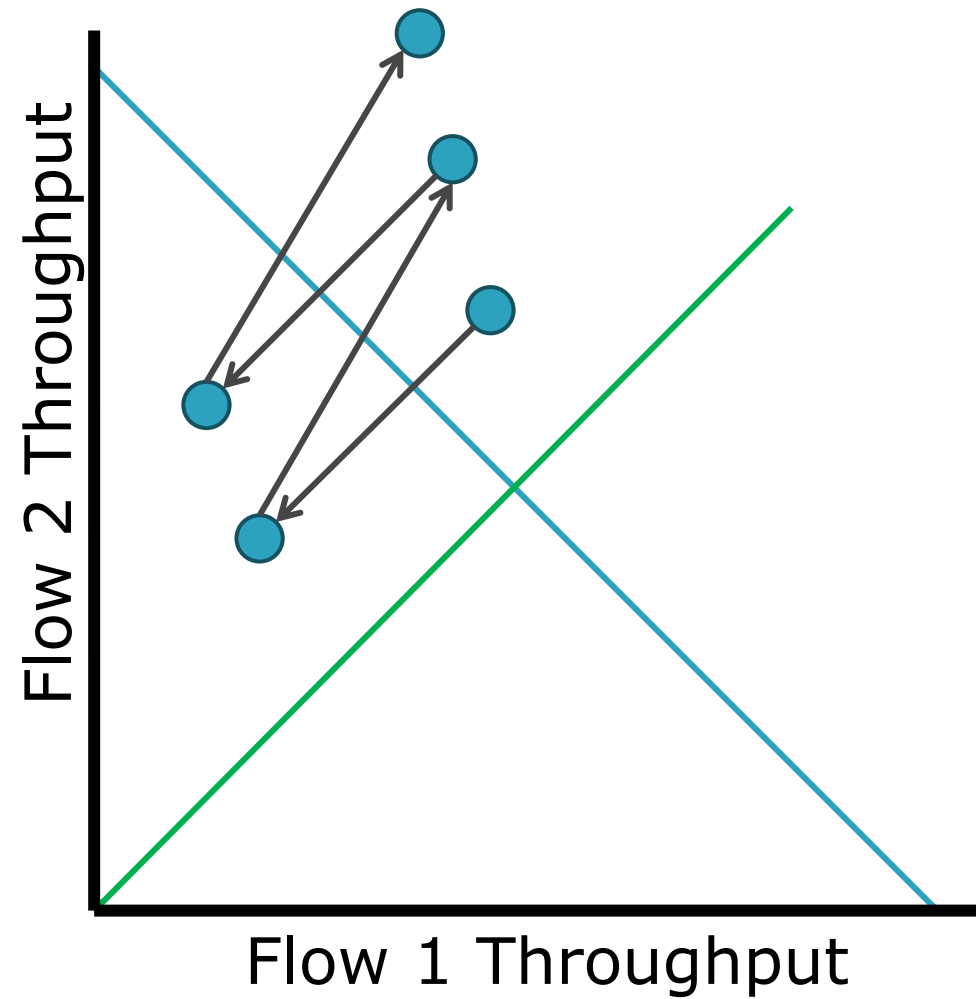
- 回想一下:TCP 是 ACK 时钟的
  - ▣ 拥堵 = 延迟 = ACK 之间的长时间等待
  - ▣ 不拥塞 = 低延迟 = ACK 快速到达
- 基本算法
  - ▣ 收到 ACK 后:增加 cwnd
  - ▣ 数据已交付,也许我们可以更快地发送
  - ▣ CWND 增长与 RTT 成正比
  - ▣ 在损失上:减少 cwnd
  - ▣ 数据丢失,必有拥塞
- 问:增减函数要用?

# Utilization and Fairness

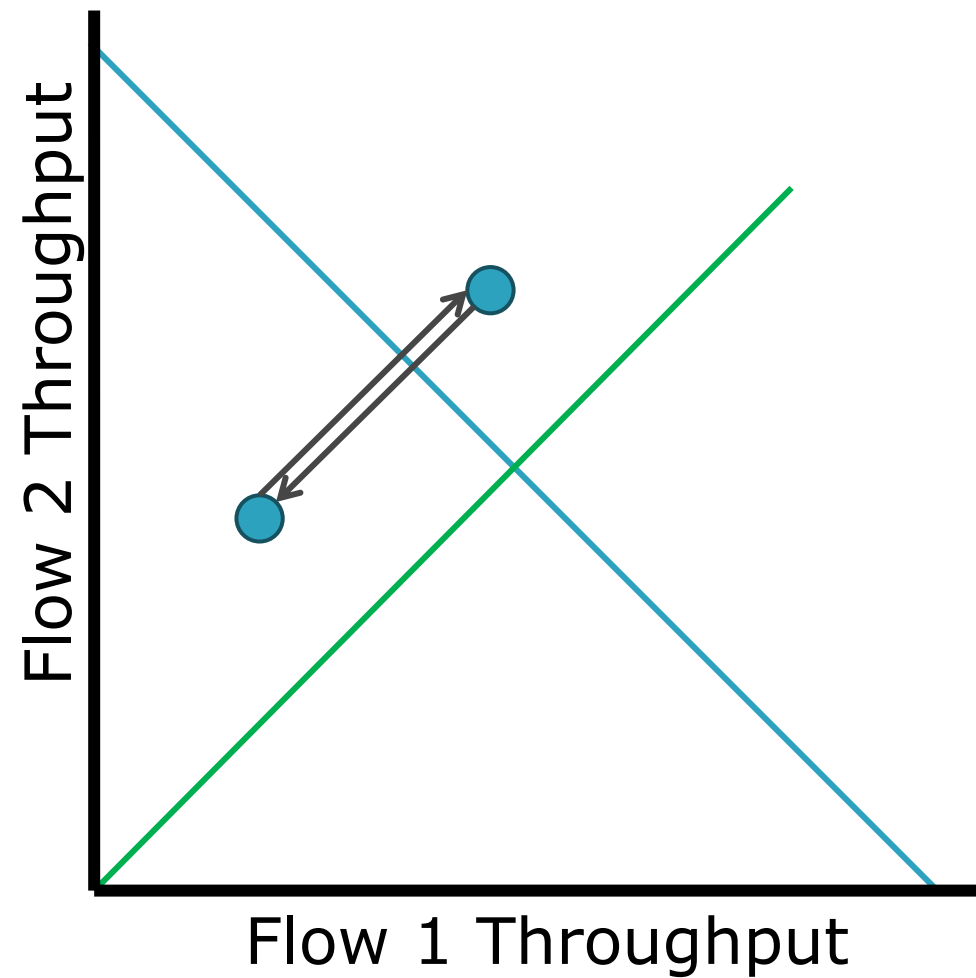


## 乘法增加,加法减少

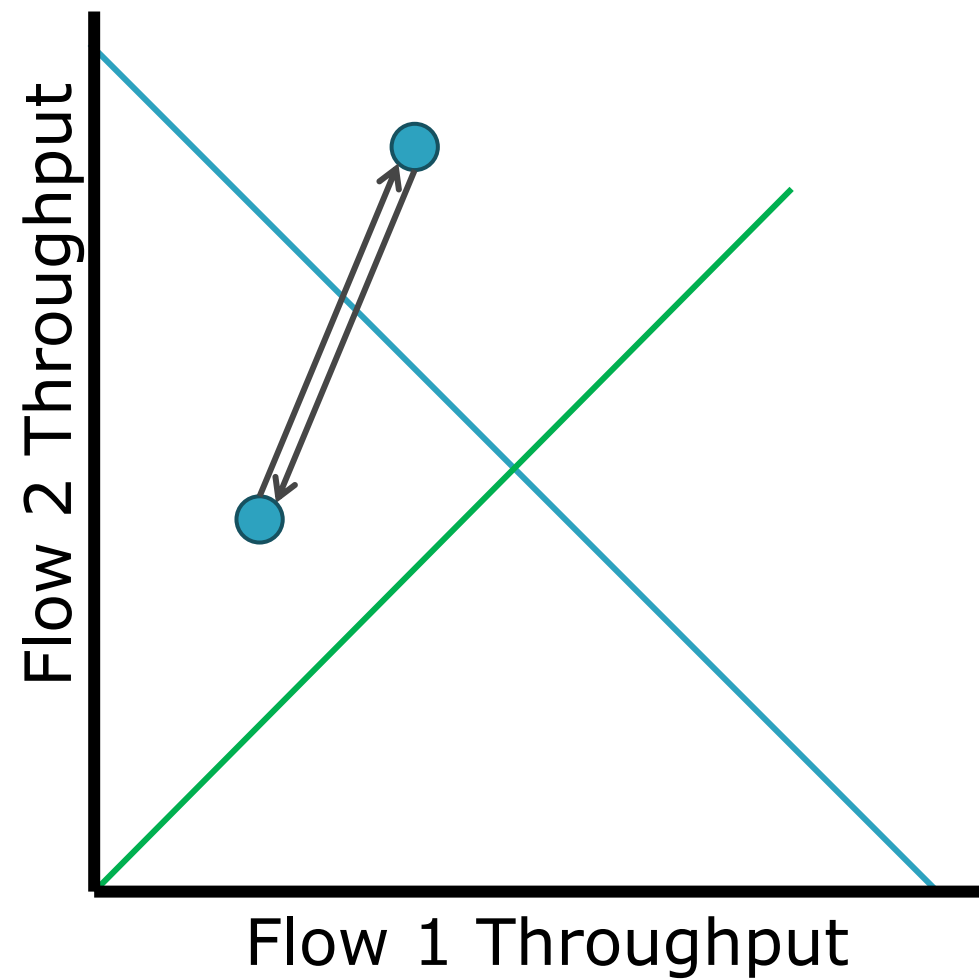
- 不稳定!
- 背离公平



- 稳定
- 但并不收敛于公平

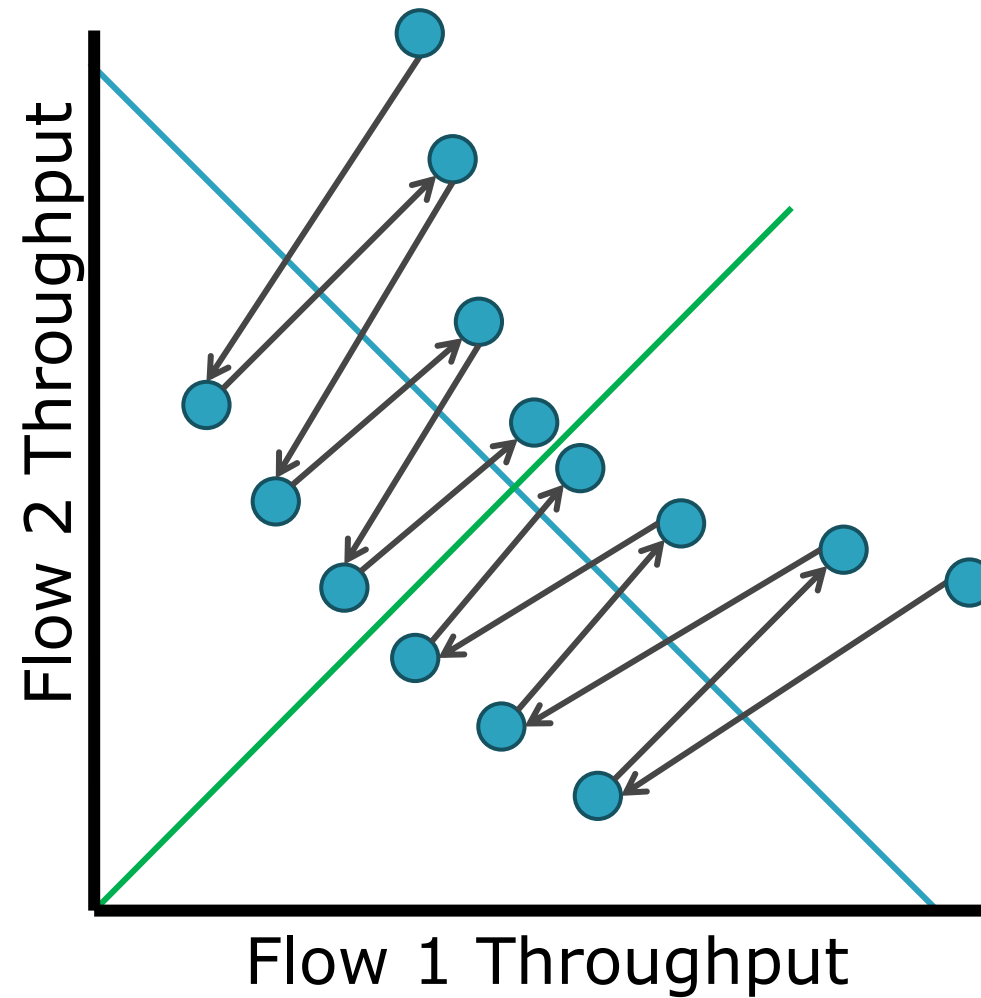


- 稳定
- 但并不收敛于公平



## 加法增加,乘法减少

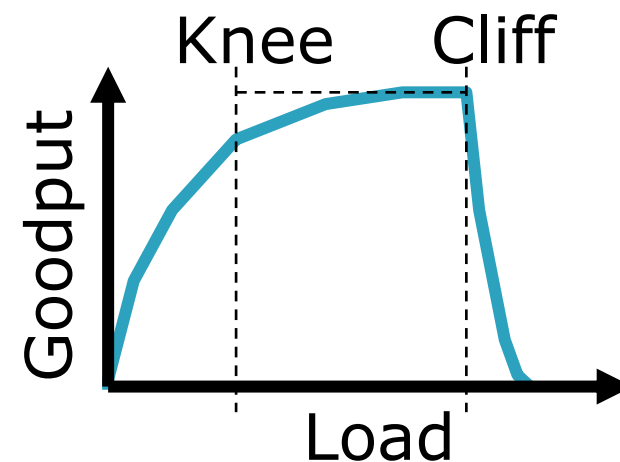
- 收敛于稳定和公平的循环
- $y=x$  对称



- 保持三个变量：
  - ▣ *cwnd*: 拥塞窗口
  - ▣ *adv\_wnd*: 接收方通告窗口
  - ▣ *ssthresh*: 阈值大小 (用于更新*cwnd*)
- 发送时使用:  $wnd = \min(cwnd, adv\_wnd)$
- 拥塞控制两个阶段
  - ▣ 慢启动 ( $cwnd < ssthresh$ )
    - 探测瓶颈带宽
  - ▣ 拥塞避免 ( $cwnd \geq ssthresh$ )
    - AIMD

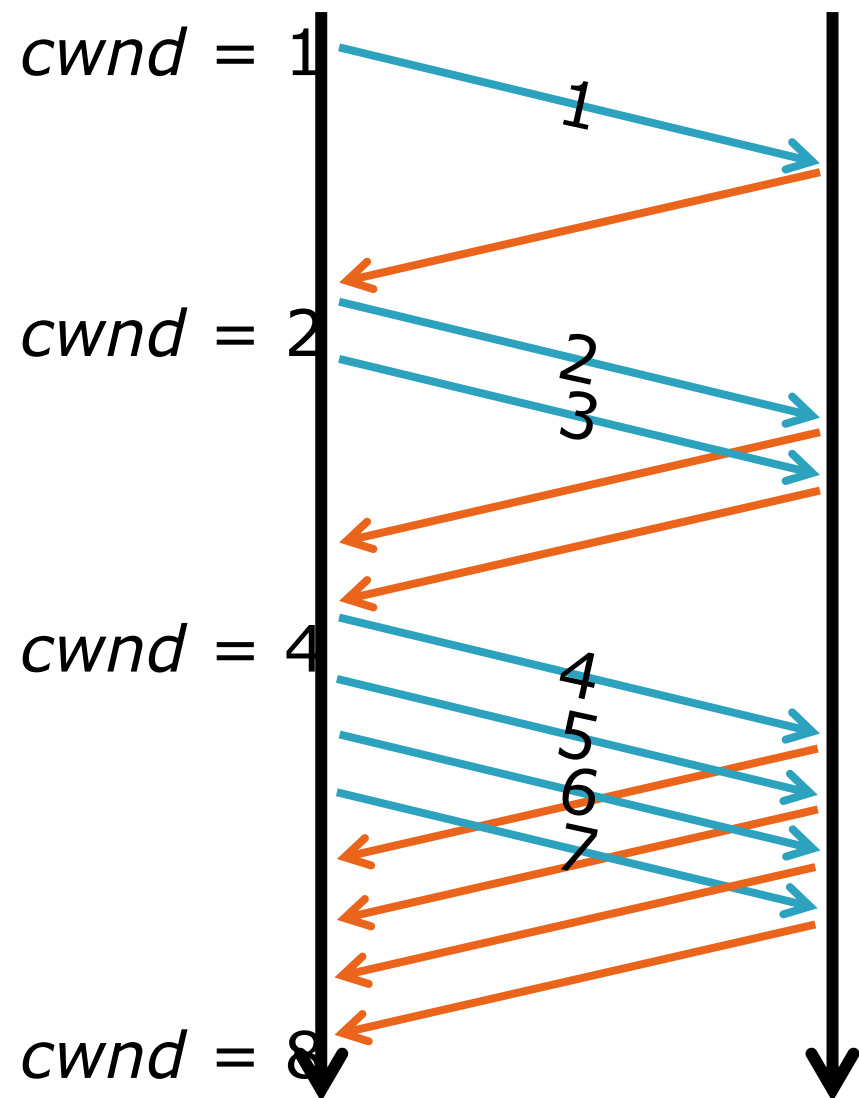
# 慢启动

- 目标：快速达到knee位置
- 在开始（或重新启动）连接时
  - ▣  $cwnd = 1$
  - ▣  $ssthresh = adv\_wnd$
  - ▣ 每一段ACKed,  $cwnd++$
- 一直持续到.....
  - ▣ 已达  $SSTHRESH$
  - ▣ 或失一包
- 慢启动其实并不慢
  - ▣ CWND 成倍增加



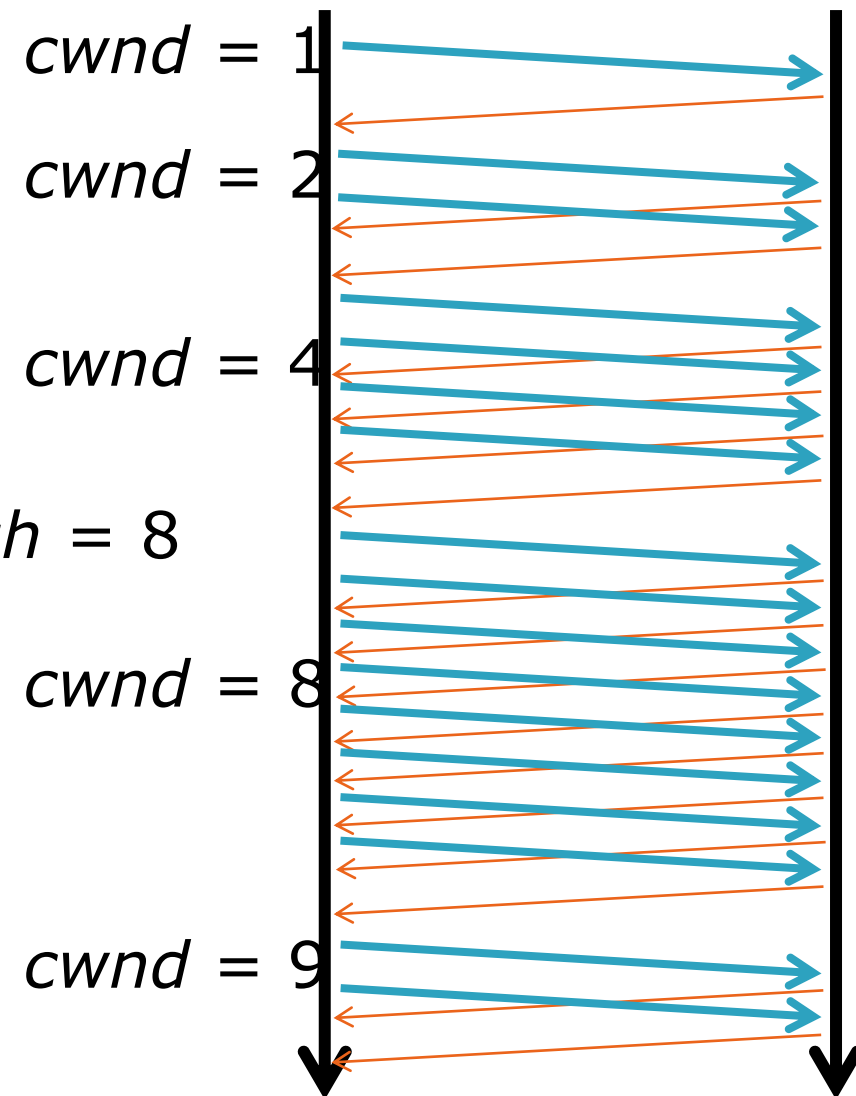
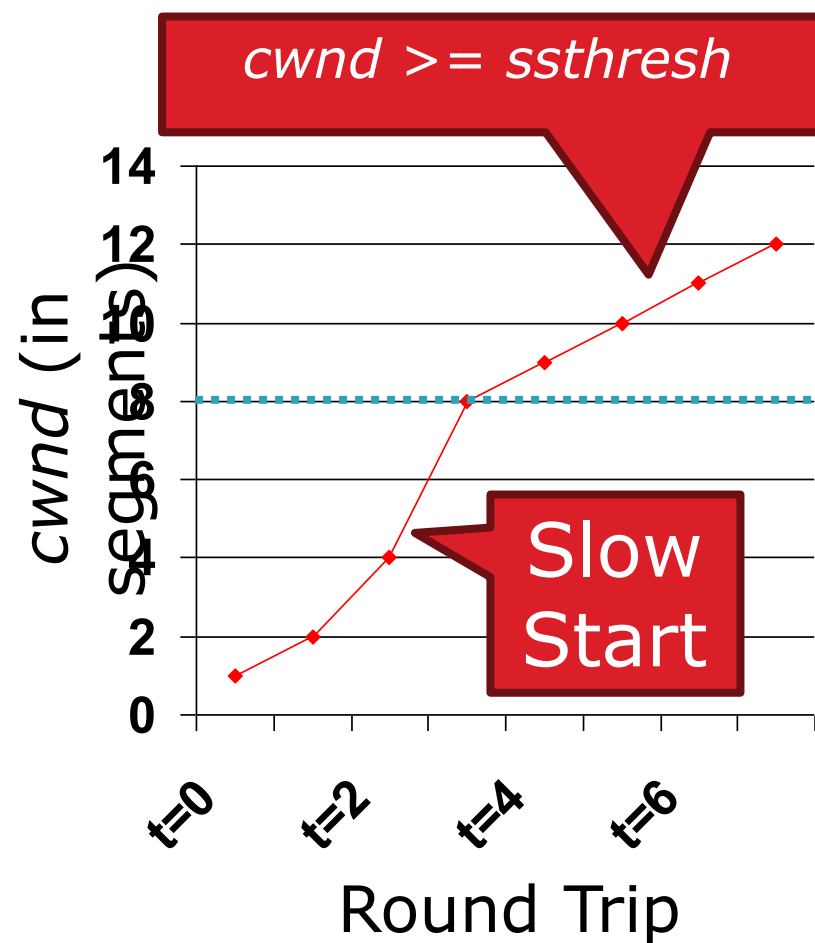
# 慢启动示例

- $cwnd$  迅速增长
- 减速, 当...
  - ▣  $cwnd \geq ssthresh$
  - ▣ 或者数据包丢失



- AIMD模式
- *ssthresh*关于knee位置的下界猜测
- **If** *cwnd*  $\geq$  *ssthresh* **then**
  - each time a segment is ACKed
  - increment *cwnd* by  $1/cwnd$  (*cwnd* +=  $1/cwnd$ ).
- 因此,只有在所有段都已确认的情况下,*cwnd* 才增加 1

# 拥塞避免示例



**Initially:**

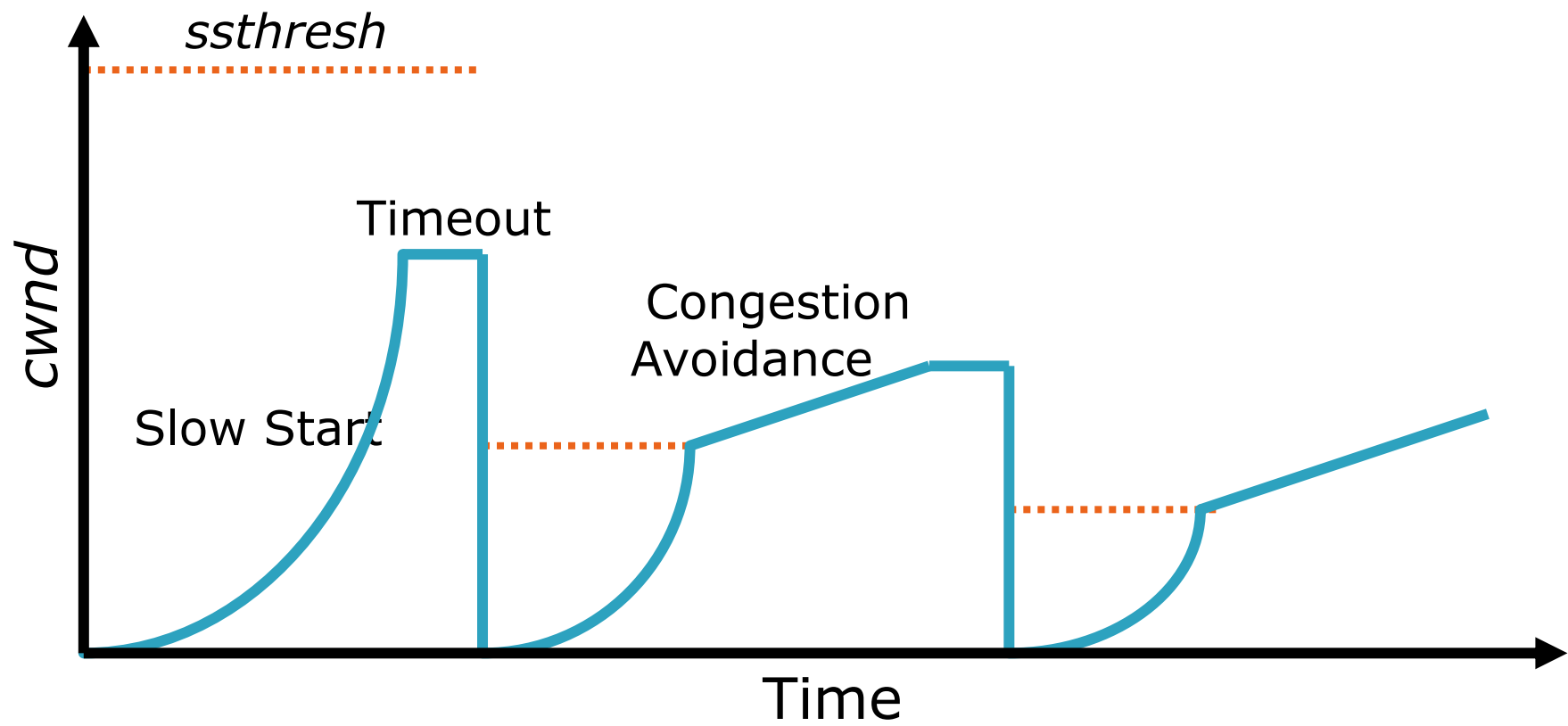
```
    cwnd = 1;  
    ssthresh = adv_wnd;
```

**New ack received:**

```
    if (cwnd < ssthresh)  
        /* Slow Start */  
        cwnd = cwnd + 1;  
    else  
        /* Congestion Avoidance */  
        cwnd = cwnd + 1/cwnd;
```

**Timeout:**

```
    /* Multiplicative decrease */  
    ssthresh = cwnd/2;  
    cwnd = 1;
```



# 目录

## Contents

1 UDP

2 TCP

3 拥塞控制

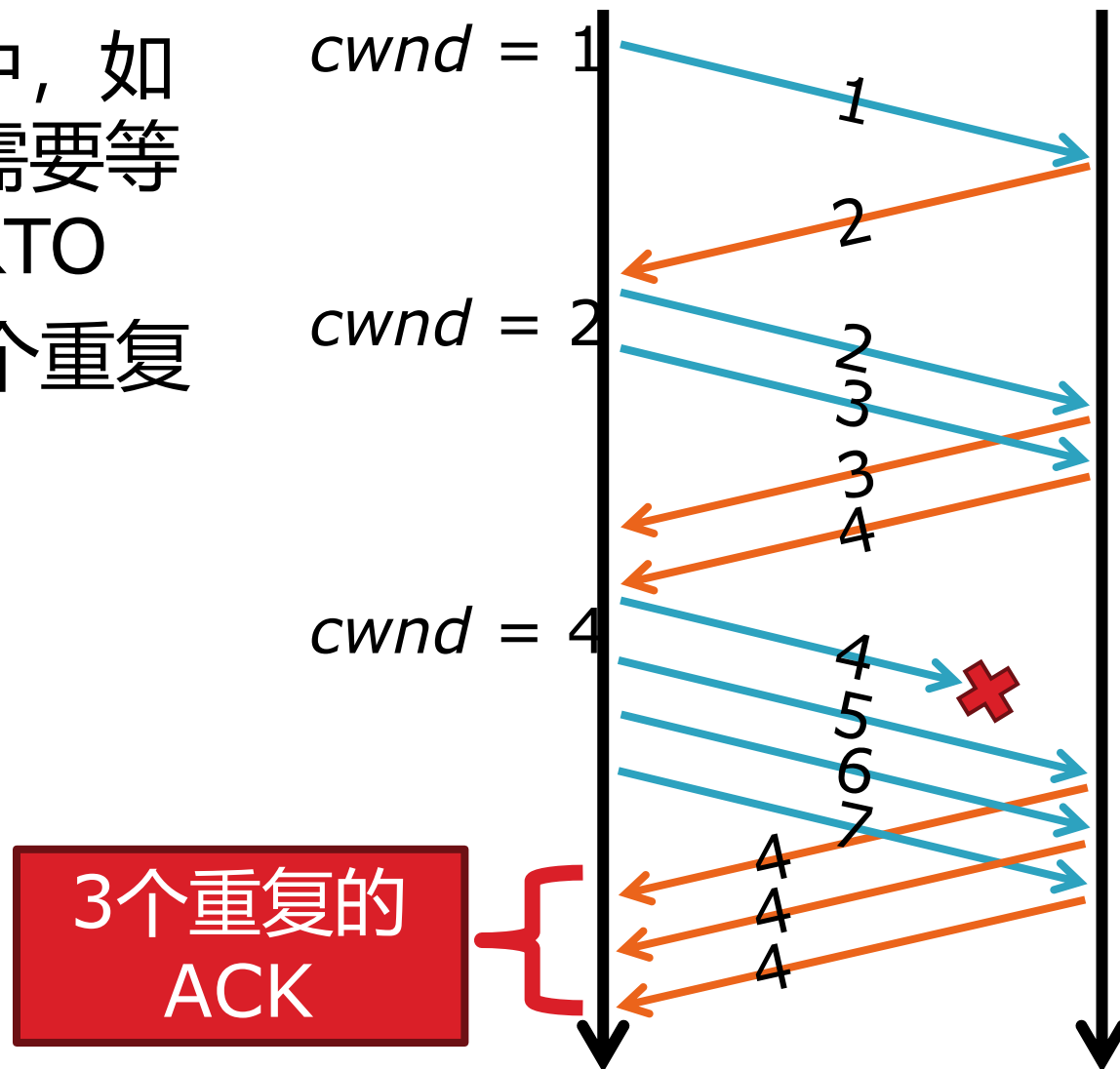
**3 TCP 的演变**

3 TCP 的问题

- 到目前为止,我们已经讨论了 TCP Tahoe
  - ▣ TCP 的原始版本
- 然而,TCP 是在 1974 年发明的!
  - ▣ 今天,TCP 有很多变体
- 早期流行的变体:TCP Reno
- 快速重传
- 恢复快

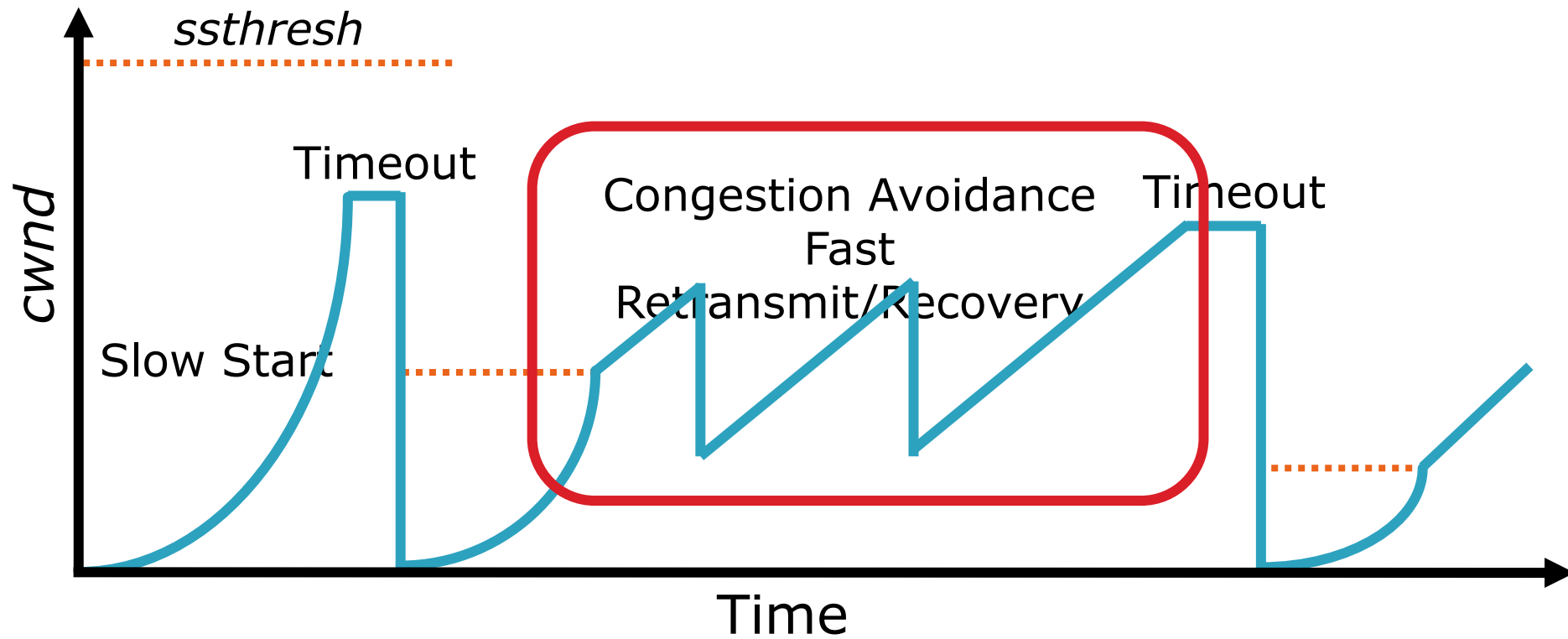
## TCP Reno:快速重传

- 问题：在Tahoe中，如果数据段丢失，需要等待很长时间直到RTO
- Reno：在收到3个重复的ACK后重传



- 快速重传后,将 `cwnd` 设置为 `ssthresh/2`
  - ▣ 即不要将 `cwnd` 重置为 1
  - ▣ 避免不必要的返回慢启动
  - ▣ 防止昂贵的超时
- 但当 RTO 到期时,仍为 `cwnd = 1`
  - ▣ 回到慢速启动
  - ▣ 表示完全没有送达数据包
  - ▣ 即拥堵必须非常严重

# 快速重传和快速恢复



- 在稳态下,  $cwnd$  围绕最佳窗口大小振荡
- TCP 总是强制丢包

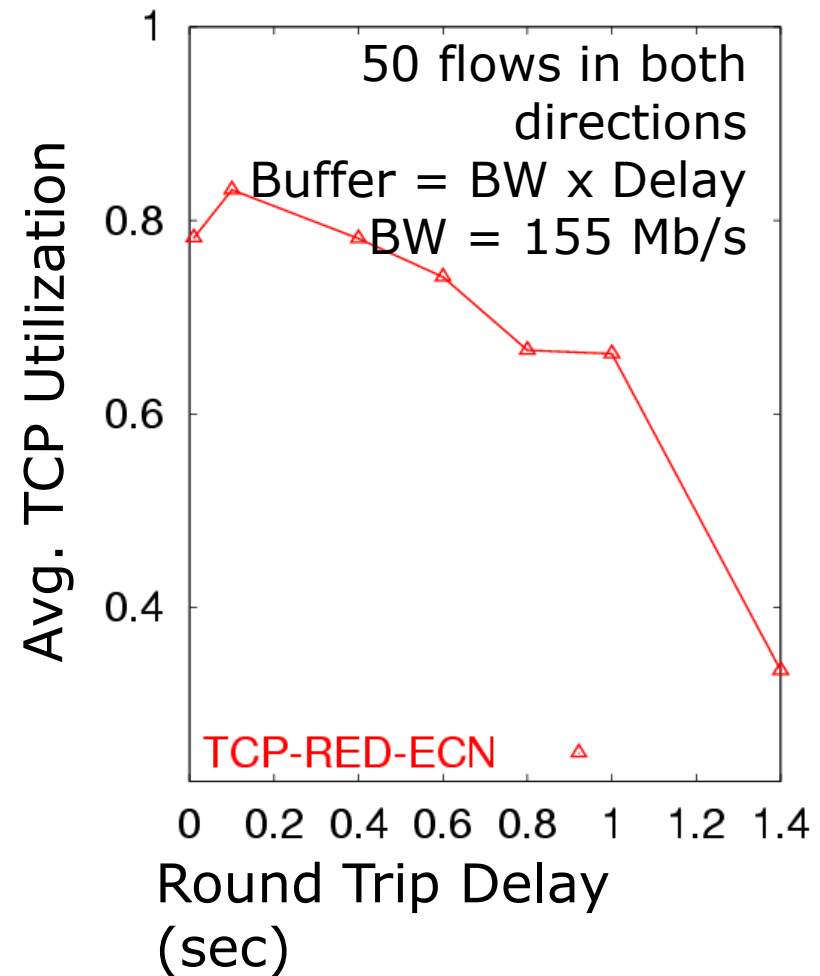
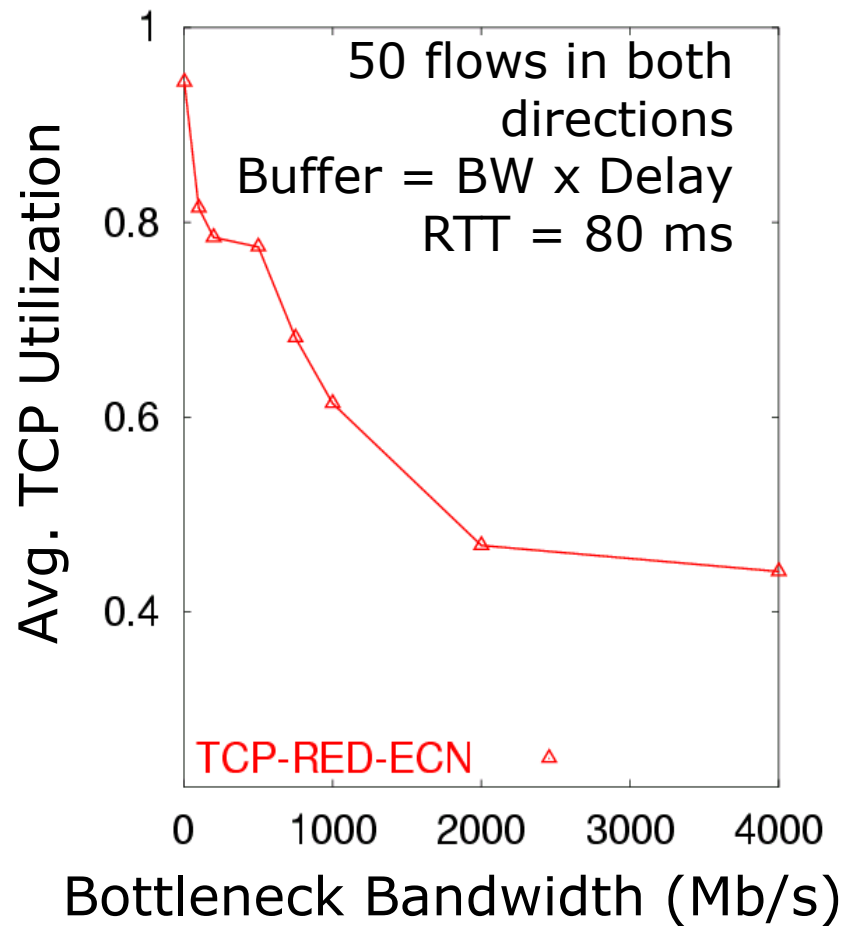
## 许多 TCP 变体.....

- Tahoe: 原版
  - ▣ AIMD 起步缓慢
  - ▣ 基于 RTT 估计的动态 RTO
- Reno: 快速重传和快速恢复
- NewReno: 改进的快速重传
  - ▣ 每个重复的 ACK 都会触发一次重传
  - ▣ 问题: > 3 个无序数据包导致病态重传
- Vegas: 基于延迟的拥堵避免
- 还有很多很多.....

- 当今最流行的变体是什么?
  - ▣ 关键问题:TCP 在高带宽延迟产品网络(如现代互联网)上性能不佳
  - ▣ 复合 TCP (Windows)
    - 基于Reno
    - 使用两个拥塞窗口:基于延迟和基于损失
    - 因此,它使用复合拥塞控制器
  - ▣ TCP CUBIC (Linux)
    - 增强 BIC(二进制增加拥塞控制)
    - 窗口大小由立方函数控制
    - 以自上次丢包以来的时间  $T$  参数化

- ❑ 关键问题: TCP 性能不佳
  - ▣ 网络的容量(带宽)很大
  - ▣ 网络的延迟 (RTT) 很大
  - ▣ 或者, 当带宽\*延迟很大时
    - $b * d =$  网络中飞行数据的最大量
    - 又名带宽延迟乘积
- ❑ 为什么TCP性能不佳?
  - ▣ 迟缓的开始和加法的增加, 收敛迟缓
  - ▣ TCP 是 ACK 时钟的
    - 即 TCP 只能在收到 ACK 时迅速做出反应
    - 大 RTT ACK 延迟, TCP 反应迟缓

# TCP Reno CC 性能不佳

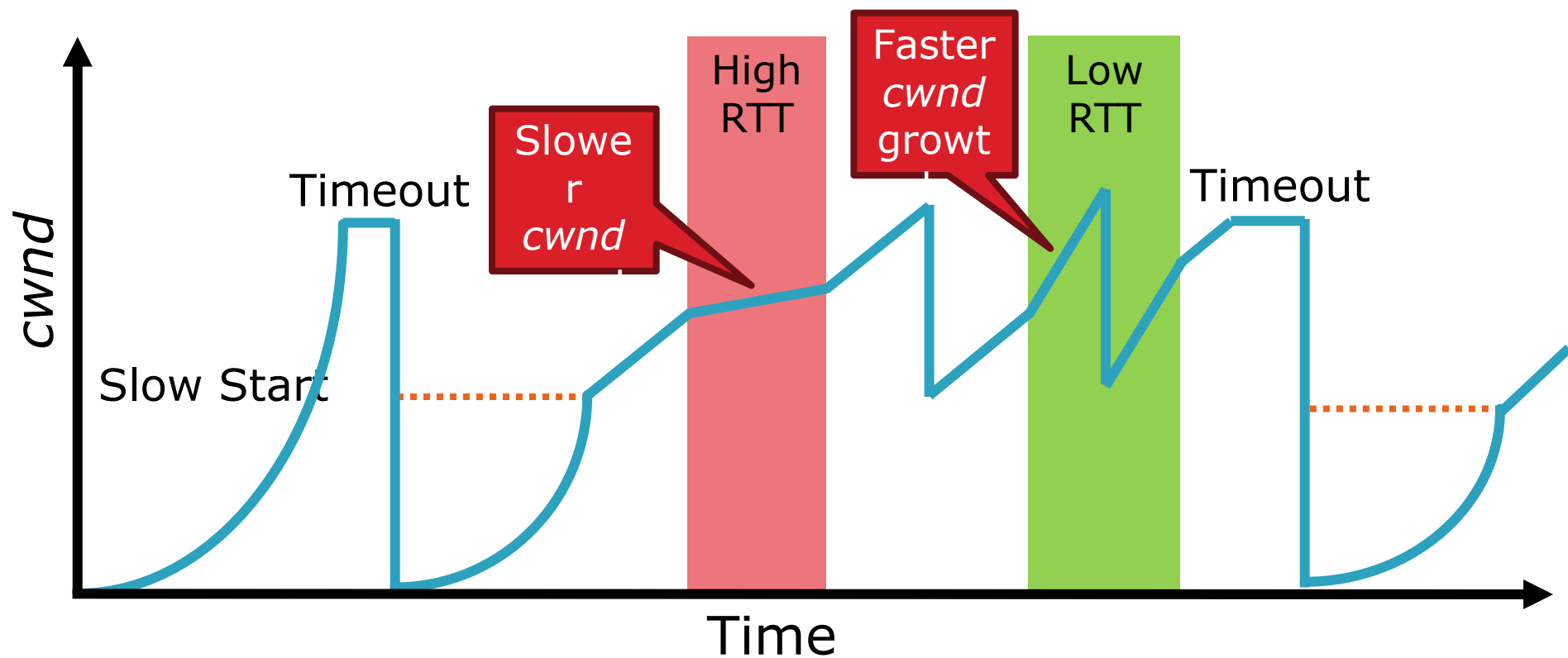


# 目标

- 窗口增长快
  - ▣ 当带宽大时,缓慢的启动和加法增加太慢
  - ▣ 想要更快地收敛
- 与其他 TCP 变体保持公平
  - ▣ 窗口增长不能太激进
- 提高 RTT 公平性
  - ▣ 当 RTT 差异很大时,TCP Tahoe/Reno 流量不公平
- 实施简单

- Windows 中的默认 TCP 实现
- 关键思想:将  $cwnd$  分为两个单独的窗口
  - ▣ 传统的基于损失的窗口
  - ▣ 新的基于延迟的窗口
- $wnd = \min(cwnd + dwnd, adv\_wnd)$ 
  - ▣  $cwnd$  由 AIMD 控制
  - ▣  $dwnd$  是延迟窗口
- 调整  $dwnd$  的规则:
  - ▣ 如果 RTT 增加,则减少  $dwnd$  ( $dwnd \geq 0$ )
  - ▣ 如果 RTT 减少,则增加  $dwnd$
  - ▣ 增减与变化率成正比

## 复合 TCP 示例



- ❑ 攻击性与 RTT 的变化相对应
- ❑ 优点:快速上升,对不同 RTT 的流量更公平
- ❑ 缺点:必须估计RTT,这是非常困难的

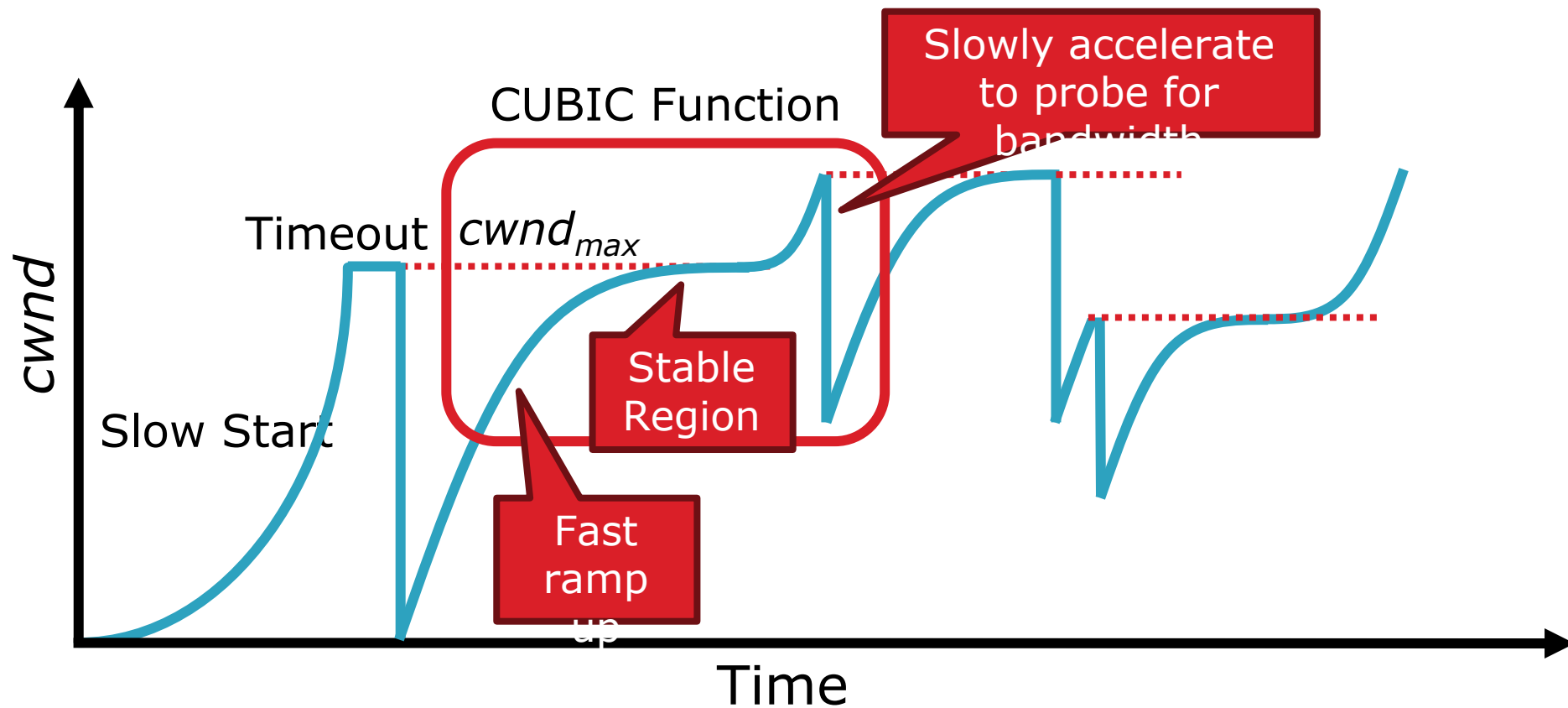
- Linux 中的默认 TCP 实现
- 以三次函数代AIMD

$$W_{cubic} = C(T - K)^3 + W_{max} \quad (1)$$

C is a scaling constant, and  $K = \sqrt[3]{\frac{W_{max}\beta}{C}}$

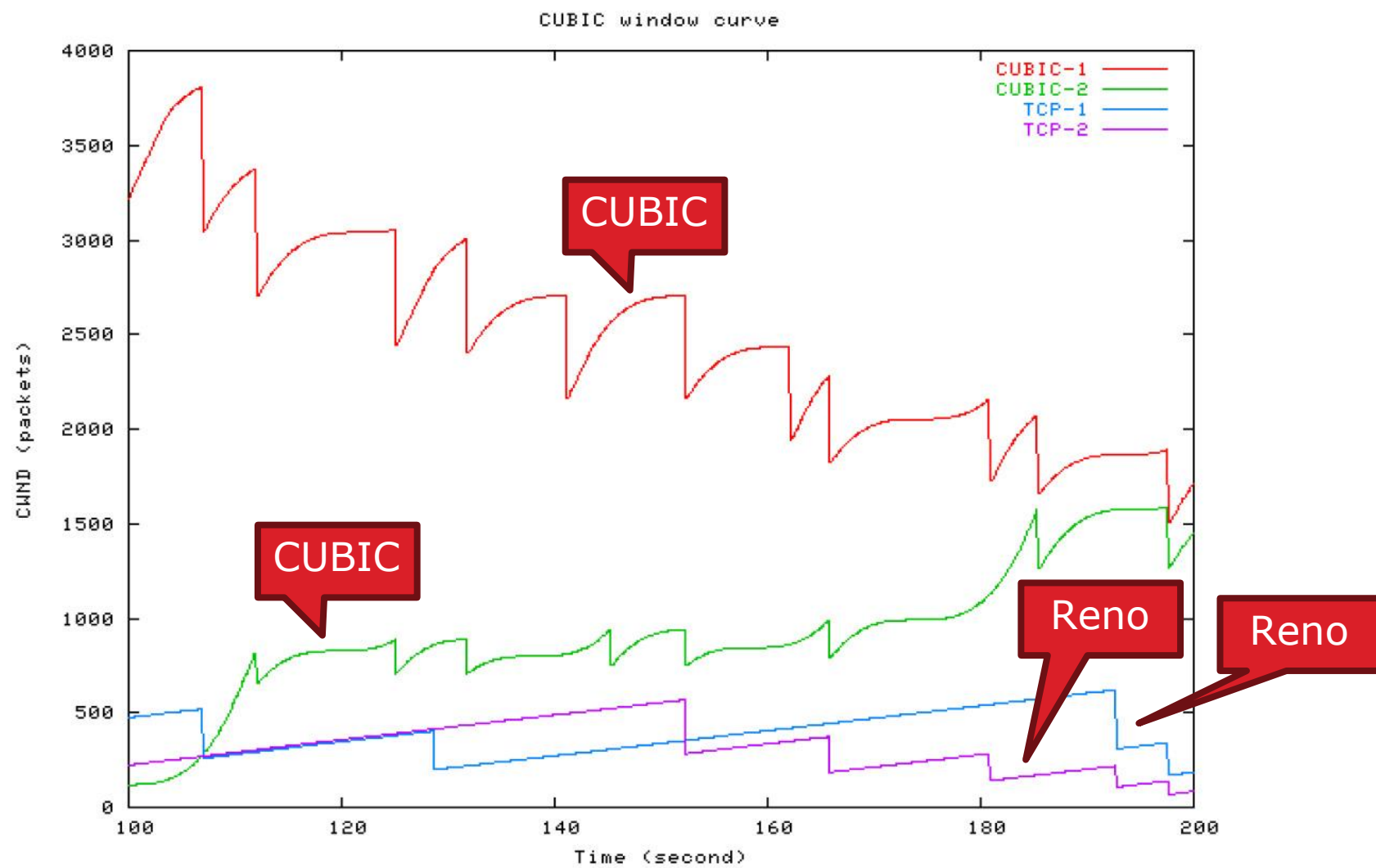
- B → 乘法增加的常数分数
- T → 自上次丢包以来的时间
- W\_max → cwnd当最后一个数据包丢弃时

# TCP CUBIC 示例



- 由于快速爬坡,减少浪费的带宽
- 区域稳定,加速缓慢,有助于保持公平
  - ▣ 快速增加比添加更激进
  - ▣ 为了公平地对待 Tahoe/Reno,CUBIC 需要不那么激进

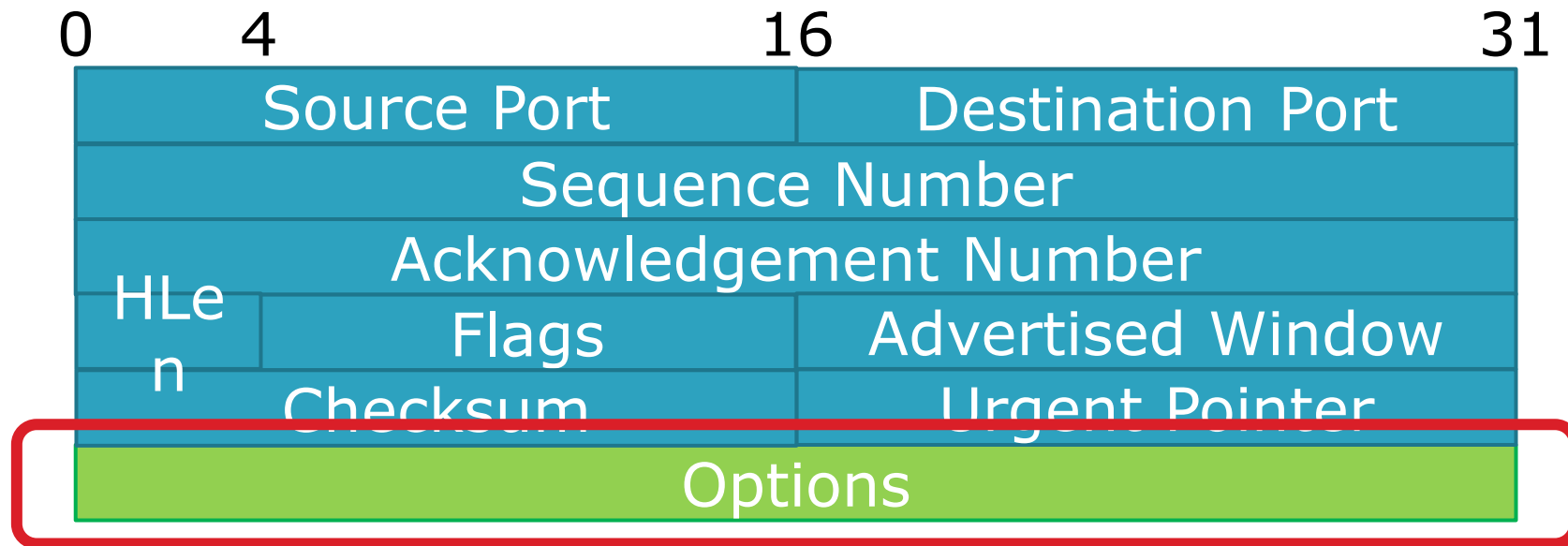
# CUBIC流模拟



- ❑ TCP 假设所有流都采用类似 TCP 的拥塞控制
  - ▣ TCP 友好或 TCP 兼容
  - ▣ 被 UDP :( 所侵犯
- ❑ 如果开发新的拥塞控制算法,它们必须是TCP友好的
- ❑ 谨防不测的互动
- ❑ 变体与同类人相得益彰
  - ▣ 不同的变体争夺资源,可能会引发不公平的病态行为

- Cerf/Kahn
  - ▣ 提供流量控制
  - ▣ 通过重传处理的拥塞
- Jacobson / Karels
  - ▣ 需要避免拥堵
  - ▣ RTT 估计关键
  - ▣ 排队理论可以帮助
- Winstein/Balakrishnan
  - ▣ TCP 是最大化目标函数
    - 公平/效率
    - 吞吐量/延迟

## 常见的 TCP 选项

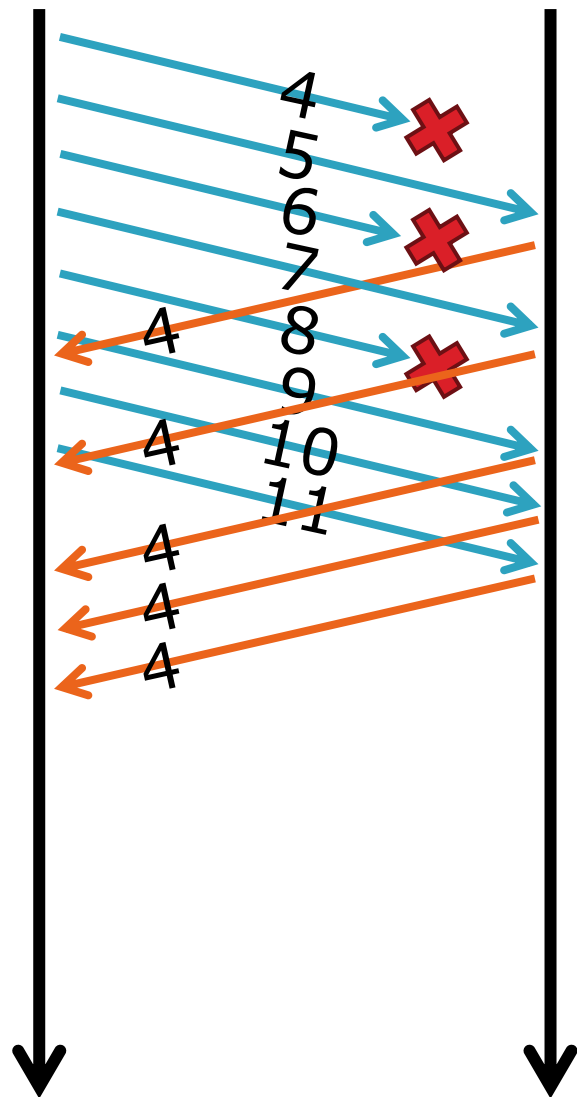


- ❑ 窗口缩放
- ❑ SACK: 选择性承认
- ❑ 最大段大小 (MSS)
- ❑ 时间戳记

- 问题：广告窗口仅为16位
  - ▣ 实际上将窗口限制在 65536B,64KB
  - ▣ 示例:1.5Mbps 链路,513ms RTT
$$(1.5\text{Mbps} * 0.513\text{s}) = 94\text{KB}$$
$$64\text{KB} / 94\text{KB} = 68\%$$
最大可能的速度
- 解决方法:引入窗口缩放值
  - ▣  $wnd = adv\_wnd \ll wnd\_scale;$
  - ▣ 最大移位为 14 位,最大 1GB 窗口

# SACK:选择性承认

- 问题:重复的 ACK 只告诉我们 1 个缺失的数据包
  - ▣ 需要多轮 dup ACK 来填补所有漏洞
- 解决方案:选择性 ACK
  - ▣ 在 TCP 标头中包含已接收的无序序列号
  - ▣ 明确地告诉发送者序列中的漏洞



## 其他常见选项

- 最大段大小 (MSS)
  - ▣ 本质上,什么是主机 MTU
  - ▣ 节省路径发现开销
- 时间戳记
  - ▣ 包裹大约是什么时候寄来的?
  - ▣ 用于防止序列号环绕
  - ▣ PAWS 算法

# 目录

## Contents

1 UDP

2 TCP

3 拥塞控制

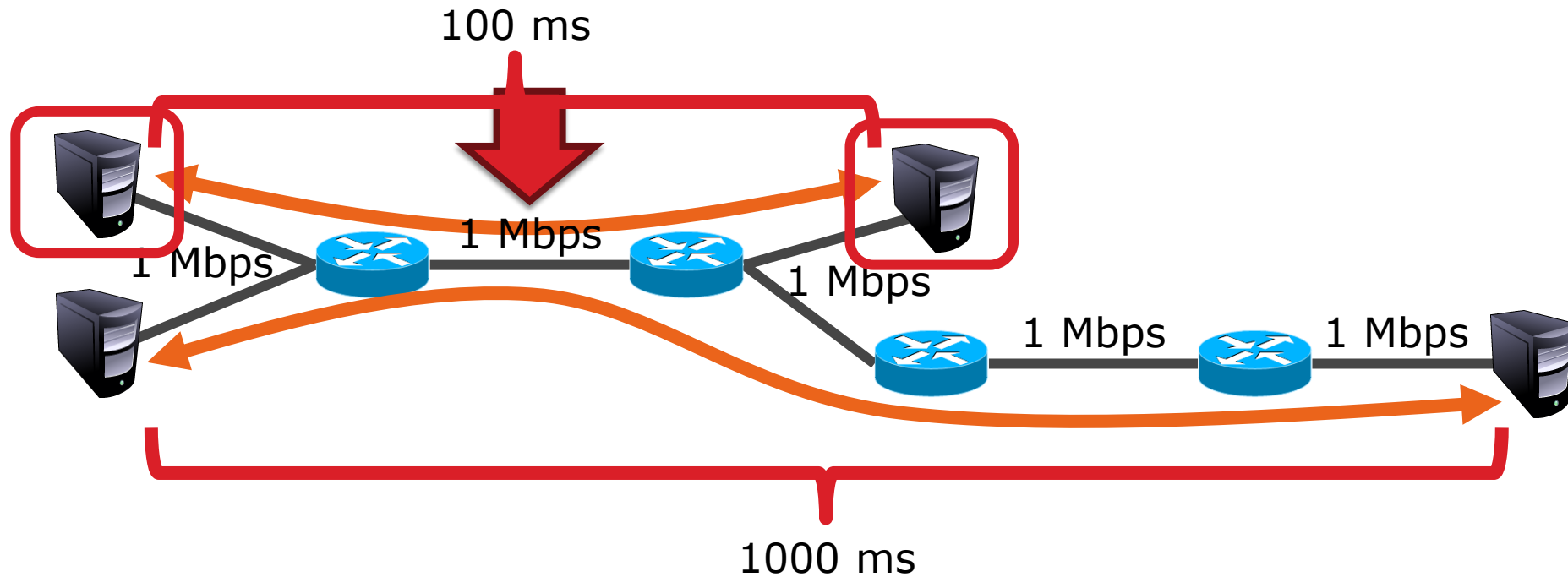
4 TCP 的演变

**5 TCP 的问题**

# TCP 的问题

- Internet 流量的绝大部分是 TCP
- 然而,协议中存在许多问题
  - ▣ 不公平
  - ▣ 流之同步
  - ▣ 性能不佳,流量小
  - ▣ 无线网络性能非常差
  - ▣ 易被拒绝服务

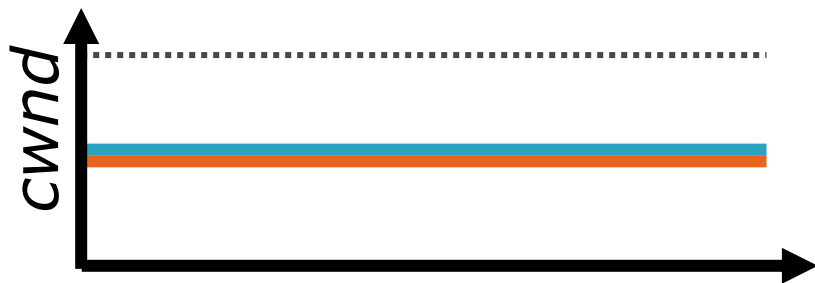
- 问题: TCP 吞吐量取决于 RTT



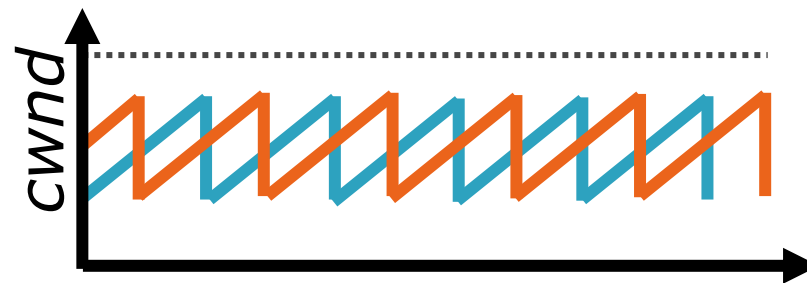
- ACK 时钟使 TCP 固有不公平
- 可能的解决办法: 保持一个单独的延迟窗口
  - ▣ 由 Microsoft 的 Compound TCP 实现

# 流动同步

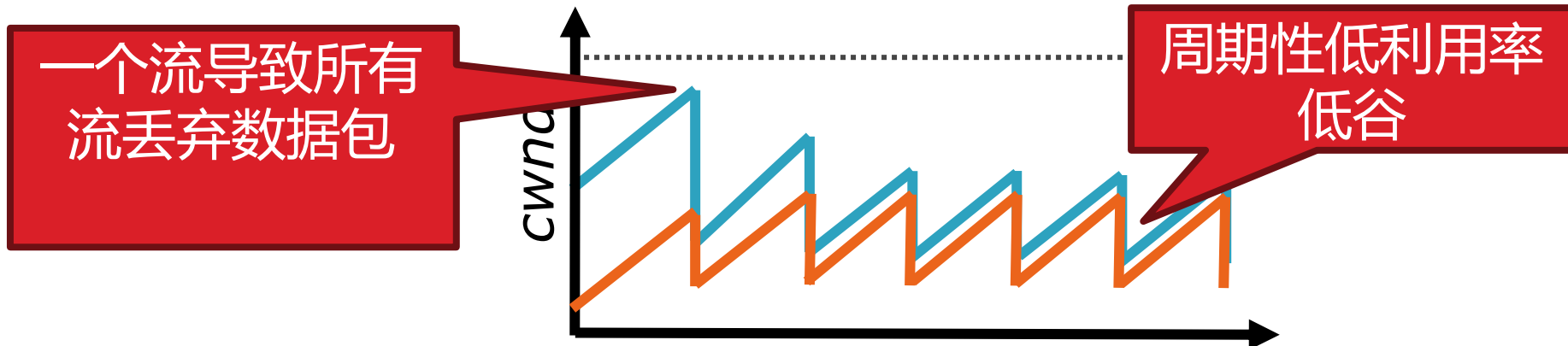
□ 理想的带宽共享



□ 振荡,但总体利用率高



□ 实际上,流动是同步的



- ❑ 问题:TCP 偏向于短流
  - ▣ 1 RTT 浪费用于连接设置 (SYN, SYN/ACK)
  - ▣ cwnd 总是从 1 开始
- ❑ 互联网流量绝大部分是短流
  - ▣ 主要是 HTTP 传输, <100KB
  - ▣ 大多数 TCP 流从不离开慢启动!
- ❑ 建议的解决方案(由 Google 驱动):
  - ▣ 增加初始 cwnd 为 10
  - ▣ TCP Fast Open:使用加密哈希来识别接收者,无需三方握手

- 问题:太浩湖和里诺假设损失=拥堵
  - ▣ 诚然,在 WAN 上,位错误非常罕见
  - ▣ 无线上虚假,干扰很常见
- TCP 吞吐量  $\sim 1/\sqrt{\text{drop rate}}$ 
  - ▣ 即使是几次干扰掉落也会杀死性能
- 可能的解决办法:
  - ▣ 打破分层,将数据链路信息推送到 TCP
  - ▣ 使用基于延迟的拥塞检测 (TCP Vegas)
  - ▣ 显式拥塞通知 (ECN)

- ❑ 问题: TCP 连接需要状态
  - ▣ 初始 SYN 在服务器上分配资源
  - ▣ 状态必须持续几分钟 (RTO)
- ❑ SYN flood: 向服务器发送足够的 SYN 以分配所有内存/熔毁内核
- ❑ 解决方案: SYN cookie
  - ▣ 想法: 不要将初始状态存储在服务器上
  - ▣ 将状态安全地插入 SYN/ACK 数据包中
  - ▣ 客户端会将状态反射回服务器

# SYN Cookies



- 客户最近真的给我寄了 SYN 吗?
  - ▣ 时间戳:新鲜度检查
  - ▣ 加密哈希:防止欺骗数据包
- 最大段大小 (MSS)
  - ▣ 通常由客户在初始 SYN 时说出
  - ▣ 服务器应存储此值.....
  - ▣ 通过他们反映客户的价值

# SYN Cookie 的实践

## □ 优点

- ▣ 有效缓解 SYN 洪水
- ▣ 兼容所有 TCP 版本
- ▣ 只需修改服务器即可
- ▣ 无需客户支持

## □ 缺点

- ▣ MSS 限制为 3 位,可能小于客户端实际 MSS
- ▣ 服务器忘记了客户端 SYN 中包含的所有其他 TCP 选项
- ▣ SACK 支持、窗口缩放等。